
Tree-Structured Synergy of Large Language Models and Bayesian Optimization for Efficient CASH

Beicheng Xu[†] Weitong Qian[†] Lingching Tung[†] Yupeng Lu[†] Bin Cui[†]

[†]School of Computer Science, Peking University, Beijing, China

{beichengxu, weitong.qian, lingchingtung}@stu.pku.edu.cn

{xinkelyp, cui.bin}@pku.edu.cn

Abstract

To lower the expertise barrier in machine learning, the AutoML community has focused on the CASH problem, which jointly automates algorithm selection and hyperparameter tuning. While traditional methods like Bayesian Optimization (BO) struggle with cold-start issues, Large Language Models (LLMs) can mitigate these through semantic priors. However, existing LLM-based optimizers generalize poorly to high-dimensional, structured CASH spaces. In this paper, we propose LB-MCTS, a trajectory-structured optimization framework that uses a Monte Carlo Tree Search tree as a shared state for algorithm selection, hyperparameter refinement, and BO-LLM proposer synergy. Within this shared state, BO provides algorithm-specific surrogate modeling for quantitative search, while the LLM exploits path-aware selective memory to generate semantic proposals and reflections. As the surrogate model improves, a reliability-aware proposer policy adaptively shifts from LLM-driven to BO-driven proposals within a unified search trajectory. Experiments on 104 AMLB datasets demonstrate that LB-MCTS consistently outperforms BO-based, LLM-based, and hybrid baselines.

1 Introduction

In recent years, machine learning has advanced in diverse domains, such as computer vision [11, 21], and recommendation systems [46, 42]. Despite these advancements, developing tailored solutions with strong performance remains a knowledge-intensive task, requiring careful selection of suitable ML algorithms and hyperparameter tuning. To lower barriers and streamline deployment, the AutoML community has introduced the Combined Algorithm Selection and Hyperparameter Optimization (CASH) problem [43] and proposed several methods [16, 12] to automate this optimization process. Among them, the dominant approach is Bayesian Optimization (BO), which builds surrogate models of the performance landscape to guide iterative evaluations. However, by treating configurations as abstract numeric vectors, pure BO solvers fail to incorporate prior knowledge, necessitating extensive evaluations to explore high-dimensional and conditional search spaces [7].

Recently, the reasoning and in-context learning (ICL) capabilities of large language models (LLMs) have made them attractive for black-box optimization [20, 48]. LLMs can serve as context-aware “experts” with domain knowledge, alleviating the limitations of BO. Early works treat the LLM as a standalone optimizer, iteratively prompting it to obtain candidates [48]. However, pure LLM optimizer suffers from inherent limitations and is unreliable in many practical scenarios [14]. This is because it does not fit a surrogate model (nor quantify uncertainty) and struggles to utilize the full optimization history with a limited context window. Therefore, attempts have been made to integrate LLMs with BO [1, 4], e.g., using LLMs to generate warm-starts or candidates for BO, or interleaving them with BO for alternating optimization. Such synergy has yielded improved performance.

Corresponding author.

However, existing LLM-based optimizers remain confined to unstructured and low-dimensional problems, leaving the highly structured and high-dimensional CASH problem unexplored. Tackling CASH with LLMs presents three critical challenges that current approaches fail to address: **C1: Efficient context utilization.** CASH features a hierarchical search space (i.e., specific hyperparameters active only for specific algorithms) where the history of one algorithm (e.g., SVM) provides negligible guidance for another (e.g., XGBoost). Yet, existing methods indiscriminately populate prompts with the best, recent, or entire optimization history. This approach introduces cross-algorithm noise, leading to low context utilization and high token costs. **C2: Exploration-exploitation trade-off.** Success in CASH demands a dual-level exploration-exploitation balance: selecting between distinct algorithms, and navigating between promising vs. unexplored regions within each algorithm’s subspace. However, existing LLM-based methods lack an explicit trade-off mechanism, leading to premature convergence and local optima. **C3: Effective synergy with BO.** Existing hybrids often neglect BO’s structured optimization trajectories, relying instead on a mere exchange of unstructured history. Furthermore, they either use static integration strategies without dynamic adaptation or rely on rigid thresholds to switch between LLM and BO, which generalizes poorly. These challenges share a common root: existing methods lack a structured state that models hierarchical CASH optimization trajectories and coherently integrates BO- and LLM-produced trajectories. Consequently, LLMs receive flat and noisy histories, and BO–LLM interaction reduces to heuristic alternation.

To address this limitation, we propose LB-MCTS, a trajectory-structured optimization framework for CASH. The key idea is to use an MCTS tree as a structured search state, where each path records a hierarchical optimization trajectory and supports BO–LLM proposer synergy through shared trajectory updates. The contributions are summarized as follows: (i) We formulate CASH as tree-structured search over optimization trajectories, enabling root-level algorithm selection and subtree-level exploration-exploitation within each algorithm’s HPO space (**C2**). (ii) We realize BO–LLM proposer synergy by unifying BO- and LLM-produced optimization trajectories in the shared tree state and dynamically selecting the proposer for configuration proposal, achieving more sample-efficient optimization than either component alone (**C3**). (iii) We introduce Selective Tuning Memory (STM), which retrieves algorithm-specific and trajectory-relevant trials from the shared tree state, allowing the LLM to reason from focused, low-noise context (**C1**). (iv) Experiments on 104 AMLB datasets show that LB-MCTS achieves the best average validation rank of 2.16, outperforming the strongest baseline RB (3.49), as well as BO-based, LLM-based, and hybrid baselines.

2 Preliminary and Related Work

CASH Problem and Traditional Methods. Let $\mathcal{A} = \{A^1, \dots, A^K\}$ be a set of K candidate algorithms, where each algorithm A^i is associated with a specific hyperparameter space Λ^i . Given a dataset partitioned into training and validation sets $(\mathcal{D}_{train}, \mathcal{D}_{val})$, CASH aims to jointly identify the optimal algorithm A^* and its configuration λ^* to maximize the validation metric $\mathcal{F}$:

$$(A^*, \lambda^*) = \underset{A^i \in \mathcal{A}, \lambda \in \Lambda^i}{\operatorname{argmax}} \mathcal{F}(A^i(\lambda, \mathcal{D}_{train}), \mathcal{D}_{val}). \quad (1)$$

The CASH problem was first introduced by Auto-WEKA [43] and solved by Bayesian Optimization (BO) [41, 15, 2], which iteratively fits a surrogate on evaluated configurations and maximizes an acquisition function to select the next trial. Many leading AutoML systems like Auto-sklearn [8], LightAutoML [44], and H2O [22] also adopt BO as the core solver for CASH. Another line of work improves BO by diversifying evaluated models [37, 33]. In addition, genetic programming [31] and the multi-armed bandits (MAB) based methods [23, 24] are also used to address the CASH problem.

LLM-based Optimizer. Recently, researchers have explored LLMs as standalone alternatives to traditional optimizers with their emergent reasoning capabilities. These approaches have been applied to diverse tasks, including chemistry [27], hyperparameter optimization [25], feature engineering [13], and molecular optimization [30]. Typical methods like LLAMBO [25] and OPRO [48] use the ICL capabilities of LLMs to iteratively propose solutions. However, these methods are mainly validated on low-dimensional black-box tasks and often require frequent LLM calls, limiting scalability and reliability. Another recent line integrates BO principles into pre-trained foundation models. FIBO [5] directly samples candidates from the posterior over the optimum, while ToSFiT [29] fine-tunes LLM with a Gaussian Process rewarding model. These methods remain largely generative-model-driven, approximating BO-style candidate generation rather than preserving an explicit numerical BO module, and thus still inherit limitations in numerical fidelity and uncertainty quantification.

In response to the limitations of pure LLM optimizers, hybrid approaches have emerged that combine LLMs and BO [35, 45]. BOPRO [1] optimizes prompt instructions with BO, whereas ADO-LLM [49] uses the LLM to generate BO candidates. SLLMBO [28] alternates between LLM-based exploitation and TPE [2] in a fixed 50:50 ratio. BORA [4] switches between LLM-based search and GP-based BO using a heuristic variance-threshold policy. Although these hybrid approaches improve upon pure LLM optimizers, they are still primarily restricted to low-dimensional settings ($D < 20$) with flat, unstructured parameter spaces. Naively extending them to the highly structured, high-dimensional CASH setting is ineffective due to the challenges **C1–C3** discussed in Section 1. Overall, effectively synergizing LLMs and BO for the CASH problem remains an open challenge.

Monte-Carlo Tree Search (MCTS). MCTS builds search trees for structured spaces by balancing exploration (trying new actions) and exploitation (refining promising actions) [19, 3, 9]. Each MCTS iteration consists of four phases: 1) **Selection:** Starting from the root, it traverses the tree to select the node for expansion. Modern variants (e.g., AlphaGo Zero [40]) employ PUCT [36] for selection. For a parent node s , the next child node s_i is selected from the set of children $\mathcal{C}(s)$ to maximize:

$$\arg \max_{s_i \in \mathcal{C}(s)} \left\{ Q(s_i) + c_{puct} \cdot P(s_i) \cdot \frac{\sqrt{N_s}}{1 + N_{s_i}} \right\}, \quad (2)$$

where $Q(s_i)$ is the mean value of s_i , $P(s_i)$ is the prior probability of visiting s_i , and $N_s(N_{s_i})$ denotes the visit count of the parent(child). The constant c_{puct} balances exploitation and exploration. 2) **Expansion:** Add a new child node with an untried action. 3) **Playout:** Conduct a simulation to yield a reward value. 4) **Backpropagation:** Propagate the reward back to update ancestral node statistics. Motivated by this, AlphaD3M [6] and MOSAIC [34] build upon MCTS to explore the pipeline search space. Recent work also combines LLMs with MCTS in sequential decision-making tasks, such as long-horizon robot task planning [50], text-based games [38], and code generation [26]. In contrast, LB-MCTS uses an MCTS tree as a shared CASH state, integrating algorithm selection, LLM-generated semantic proposals, and BO-based optimization into unified optimization trajectories.

3 Method

In this section, we present LB-MCTS, a trajectory-structured optimization framework that uses the MCTS search tree to model hierarchical CASH trajectories and synergize LLM and BO proposers. We first formulate the tree-structured search process, then describe algorithm selection, BO/LLM-based configuration proposal, and the reliability-aware proposer selection policy.

3.1 Tree Structured Search for CASH

We model CASH problem as a sequential decision-making process. As shown in Figure 1, the search space is organized into a tree structure containing three types of nodes:

- **CASH node** (root s_{root}): represents the initial state of the CASH problem. It encompasses all candidate ML algorithms and their respective hyperparameter spaces.
- **Algo nodes** (s_{algo}): immediate children of the root. Each represents the hyperparameter optimization problem for a specific algorithm A^i in the subspace Λ^i .
- **HP nodes** (s_{hp}): all descendants of an Algo node. Each HP node represents a concrete configuration $\lambda_{s_{\text{hp}}}$ of the algorithm and its validation metric $y_{s_{\text{hp}}}$.

Each node s maintains three key statistics: the visit count N_s , the cumulative reward R_s , and the subtree best performance $y_s^{\max}$, i.e., the best validation metric observed so far in the subtree rooted at s . After that, we define actions as transitions between nodes, yielding three action types:

- **Algorithm Selection Action** ($s_{\text{root}} \xrightarrow{a_{\text{sel}}} s_{\text{algo}}$): select an algorithm $A^i \in \mathcal{A}$ to be optimized, constraining the search to its specific hyperparameter subspace.
- **Initialization Action** ($s_{\text{algo}} \xrightarrow{a_{\text{init}}} s_{\text{hp}}$): generates the initial configuration λ_0 for the selected algorithm, serving as an initialization of an optimization trajectory.
- **Optimization Action** ($s_{\text{hp}} \xrightarrow{a_{\text{opt}}} s'_{\text{hp}}$): proposes a new configuration λ_{t+1} based on a basic configuration, aiming to refine the performance.

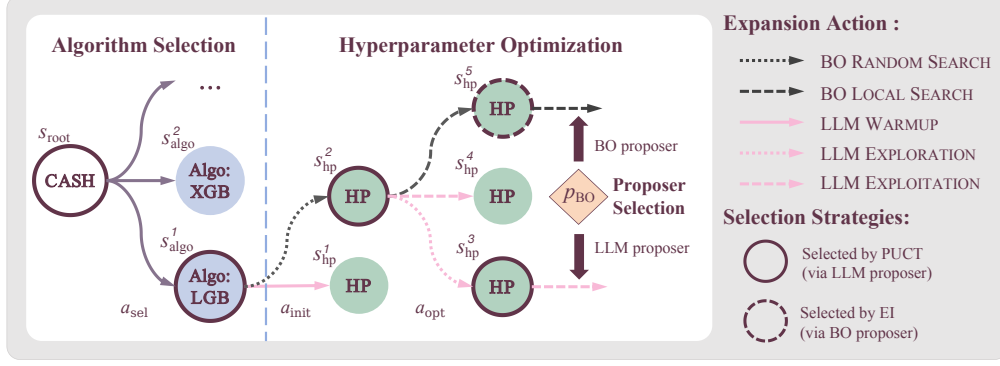


Figure 1: Tree-structured search of LB-MCTS; see Appendix A.4 for a concrete case.

We define an **optimization trajectory** as a root-originated sequence of nodes and actions. *The trajectory structurally captures the process of choosing an algorithm, initializing a configuration, and iteratively refining its hyperparameter values.* CASH is thus reformulated as a **sequential decision-making process** to find the optimal trajectory that maximizes the validation performance of the final configuration. The process can be iteratively solved by MCTS introduced in Section 2. In our setting, Expansion represents executing an action (a_{init} or a_{opt}) to propose a configuration via LLM/BO. Payout evaluates the proposed configuration to obtain its validation metric y_{new} . Finally, Backpropagation propagates this value up the trajectory: for each ancestral node s , it computes a local relative-improvement reward $r_s = [y_{\text{new}} - y_s^{\max}]_+ / |y_s^{\max}|$ before the subtree-best update, where $[x]_+ = \max(x, 0)$. It then updates the visit count ($N_s \leftarrow N_s + 1$), the cumulative reward ($R_s \leftarrow R_s + r_s$), and the subtree best ($y_s^{\max} \leftarrow \max(y_s^{\max}, y_{\text{new}})$) to guide future iterations.

Surrogate model for each algorithm. We maintain a separate Gaussian Process (GP) [47] surrogate model f_i for each algorithm A^i . Importantly, f_i is trained on the local dataset $\mathcal{D}_i = \{(\lambda_{s_{\text{hp}}}^k, y_{s_{\text{hp}}}^k)\}_{k=1}^{N_i}$, comprising all N_i configurations and their performance collected within the subtree rooted at s_{algo}^i . This per-algorithm modeling exploits disjoint hyperparameter spaces to reduce dimensionality, enhancing precision compared to a single high-dimensional global surrogate [23]. We define the GP covariance function $k(\lambda, \lambda')$ as a mixed kernel, with a Matérn-5/2 kernel (k_ν) on continuous dimensions and a Hamming kernel (k_{cat}) on categorical dimensions:

$$k(\lambda, \lambda') = k_\nu(\lambda_{\text{cont}}, \lambda'_{\text{cont}}) \times k_{\text{cat}}(\lambda_{\text{cat}}, \lambda'_{\text{cat}}), \quad (3)$$

3.2 Algorithm Selection

In each MCTS iteration, the Selection phase starts at s_{root} and chooses one of its K child Algo node, which directly corresponds to selecting an algorithm $A^i \in \mathcal{A}$ for CASH. Concretely, we utilize PUCT in Equation (2) and the estimated mean value of each Algo node s_{algo}^i is

$$Q(s_{\text{algo}}^i) = \tilde{y}_{s_{\text{algo}}^i}^{\max} \left(1 + R_{s_{\text{algo}}^i} / N_{s_{\text{algo}}^i} \right), \quad (4)$$

where $\tilde{y}_{s_{\text{algo}}^i}^{\max}$ is the normalized best validation performance in its subtree (min-max normalized using the global range of observed metric scores), and $R_{s_{\text{algo}}^i} / N_{s_{\text{algo}}^i}$ is the average local relative-improvement rate. This multiplicative form treats the mean value Q as an estimated performance potential, favoring algorithms with both high incumbent performance and continuous improvement tendency.

Algorithm Prior. We compute a PUCT prior score for each algorithm based on surrogate predictions. Specifically, for each algorithm A^i , we uniformly sample n_s configurations from Λ^i , predict their performance using the corresponding surrogate f_i , and compute the normalized mean predicted score $\bar{v}_i$. The prior distribution is then given by a softmax over these scores: $P(s_{\text{algo}}^i) = \frac{\exp(\bar{v}_i)}{\sum_{k=1}^K \exp(\bar{v}_k)}$.

3.3 Hyperparameter Optimization

Once the root selects an algorithm A^i , the task reduces to an algorithm-specific HPO subproblem: finding $\lambda \in \Lambda^i$ with best performance. During subsequent phases, LB-MCTS integrates two complementary configuration proposers: BO for quantitative search, and LLM for semantic proposals.

3.3.1 BO proposer

The BO proposer directly generates a new node by selecting the best candidate from a sampled pool.

Candidate Generation. We employ two sampling strategies to generate a pool of candidate configurations $\mathcal{P}$. (i) Random sampling draws candidates randomly from the search space Λ^i . (ii) Local sampling treats every HP node s_{hp} in the subtree of s_{algo}^i as a basic configuration and samples candidates from its neighborhood (i.e., via small perturbations of configuration in s_{hp}).

Selection & Expansion. We then rank all candidates in $\mathcal{P}$ using the Expected Improvement (EI) [18] acquisition function based on the surrogate f_i , and select the candidate λ_{new} with the highest score. The expansion type depends on the source of λ_{new} : If it originates from random sampling, it is treated as an initialization action (e.g., $s_{\text{algo}}^1 \xrightarrow[\text{BO RANDOM SEARCH}]{a_{\text{init}}} s_{\text{hp}}^2$), expanding directly from the Algo node s_{algo}^i ; If it originates from local sampling, it is treated as an optimization action (e.g., $s_{\text{hp}}^2 \xrightarrow[\text{BO LOCAL SEARCH}]{a_{\text{opt}}} s_{\text{hp}}^5$), expanding as a child of the basic configuration node.

3.3.2 LLM proposer

Following the algorithm selection, the LLM proposer continues the Selection phase using PUCT until a leaf HP node is reached. It then expands by querying the LLM to generate a new configuration.

Selection. The LLM proposer extends the Selection by recursively applying PUCT (Equation (2) and Equation (4)) within the algorithm subtree. The traversal proceeds until it reaches a node that LLM has not fully expanded, e.g., s_{hp}^3 in Figure 1. We assign a uniform prior $P(s_{\text{hp}})$ to all child nodes here.

Expansion via ICL. Given a selected leaf node s_{base} with a basic configuration λ_{base} , the LLM proposer executes Expansion by querying the LLM to propose a new configuration through adjusting λ_{base} . We construct a structured prompt comprising four key components: (i) the *task description* ψ (dataset description and optimization metric), (ii) the *Selective Tuning Memory* $\mathcal{M}_i(s_{\text{base}})$, containing informative historical trials from algorithm A^i , (iii) the basic configuration λ_{base} , and (iv) an optimization directive $d \in \{\text{WARMUP}, \text{EXPLORATION}, \text{EXPLOITATION}\}$. Note that (ii) and (iii) are empty for an initialization expansion when s is an Algo node. Formally, the LLM generates a reasoning chain $\mathcal{R}$ followed by a new configuration λ_{new} :

$$(\mathcal{R}, \lambda_{\text{new}}) = \text{LLM}(\psi, \mathcal{M}_i(s_{\text{base}}), \lambda_{\text{base}}, d). \quad (5)$$

The output λ_{new} is added as a new child node $s_{\text{hp}}^{\text{new}}$ of s_{base} . Crucially, the directive d explicitly governs the search mode: WARMUP provides a high-quality initial point when expanding an Algo node (e.g., $s_{\text{algo}}^1 \xrightarrow[\text{LLM WARMUP}]{a_{\text{init}}} s_{\text{hp}}^1$). EXPLOITATION instructs the LLM to locally refine λ_{base} for immediate gains (e.g., $s_{\text{hp}}^2 \xrightarrow[\text{LLM EXPLOITATION}]{a_{\text{opt}}} s_{\text{hp}}^4$); and EXPLORATION encourages bold adjustments to probe unknown regions (e.g., $s_{\text{hp}}^2 \xrightarrow[\text{LLM EXPLORATION}]{a_{\text{opt}}} s_{\text{hp}}^3$). An Algo node is regarded as a leaf before it has generated $n_w = 3$ initial configurations via WARMUP. An HP node is considered fully expanded by the LLM only after both EXPLORATION and EXPLOITATION have been executed once; otherwise, it remains a leaf node. Detailed prompt template and example are shown in Appendix B.1 and B.4.

Reflection after payout. Upon generation, the new configuration λ_{new} is evaluated on the training/validation split to obtain the performance metric y_{new} . To bridge the gap between numerical rewards and the LLM’s semantic reasoning, we subsequently incorporate a reflection mechanism inspired by Reflexion [39]. Excluding the WARMUP expansion of Algo nodes, we prompt LLM to analyze the evaluation result by comparing it against λ_{base} and configurations in memory $\mathcal{M}_i(s_{\text{base}})$:

$$\rho_{\text{new}} = \text{LLM}(\psi, \mathcal{M}_i(s_{\text{base}}), \lambda_{\text{base}}, y_{\text{base}}, \lambda_{\text{new}}, y_{\text{new}}). \quad (6)$$

ρ_{new} summarizes what worked or failed and provides instructions for future expansion, which is stored in the new HP node $s_{\text{hp}}^{\text{new}}$. Prompt template and example are detailed in Appendix B.2 and B.4.

Selective Tuning Memory. Efficiently utilizing optimization history is crucial for ICL. Therefore, we propose the Selective Tuning Memory (STM) mechanism. Given their disjoint search spaces, histories from different algorithms provide no mutual benefit. Thus, we first isolate the optimization history so that the memory $\mathcal{M}_i(s_{\text{base}})$ only contains trials from the same algorithm subtree as s_{base} . After

that, we view each transition in the subtree (from a parent node s_p to a child s_c) as an “optimization attempt” $e = (\lambda_p, y_p, \lambda_c, y_c, \rho_c)$, capturing the modification from the parent configuration λ_p to the child λ_c , their performances, and reflection ρ_c . For nodes expanded through BO and WARMUP lacking LLM reflection, reflections are programmatically generated via fixed rules (Appendix B.3).

Given an HP node with configuration λ_{base} to expand, our goal is to retrieve historical attempts that are both relevant (similar starting point) and instructive (high resulting performance y_c). The similarity between a historical attempt’s starting configuration λ_p and λ_{base} is measured through the algorithm’s GP covariance kernel $k(\lambda_p, \lambda_{\text{base}})$ in Equation (3), which jointly captures continuous-categorical similarity. For initialization attempts with an Algo node parent, we assign a similarity of $-\infty$. To balance relevance and performance, we compute the Pareto frontier over all historical attempts in the algorithm’s subtree based on $(k(\cdot, \lambda_{\text{base}}), y_c)$, selecting the non-dominated attempts as **Global Memory** $\mathcal{M}_{\text{global}}$. Complementing this, we include **Local Memory** $\mathcal{M}_{\text{local}}$, comprising the sequence of attempts along the ancestral path from the algorithm root to s_{base} . This preserves the optimization trajectory context. The final memory is the union $\mathcal{M}_i(s_{\text{base}}) = \mathcal{M}_{\text{global}} \cup \mathcal{M}_{\text{local}}$, providing both high-quality, relevant references and coherent trajectory.

LLM proposer workflow. Overall, the LLM proposer follows PUCT-based base-node selection, ICL-based expansion, evaluation, reflection, and backpropagation; see Appendix A.5 (Figure 8).

3.3.3 Dynamic Proposer Selection

To synergize the complementary strengths of semantic reasoning and quantitative modeling, LB-MCTS dynamically selects either the LLM or BO proposer at each HPO step. Our insight is that BO becomes increasingly reliable as data accumulates and the surrogate model matures, whereas LLMs may excel in the early stages with prior knowledge. Thus, for each algorithm A^i , we maintain a BO-selection probability p_{BO}^i based on the generalization capability of its surrogate f_i . Specifically, given the algorithm-specific dataset, we perform k -fold cross-validation, training f_i on training folds and computing the Kendall’s rank correlation coefficient (τ^i) between predicted and true rankings on validation folds to quantify how well f_i can capture the relative order of configurations. This correlation is rescaled to $[0, 1]$ as a probability: $p_{\text{BO}}^i = \max(\epsilon, \frac{\tau^i + 1}{2})$, where a small constant ϵ (e.g., 0.05) ensures the BO proposer remains active. We initialize the BO probability p_{BO}^i to ϵ and update it every k observations (e.g., $k = 5$). In each iteration, the BO proposer is selected with probability p_{BO}^i and the LLM proposer with probability $1 - p_{\text{BO}}^i$. This mechanism ensures a smooth transition from early LLM-driven search to BO-driven search as the surrogate model becomes accurate.

3.4 Algorithm Summary

The procedure of LB-MCTS is summarized in Appendix A.2. We run MCTS iteratively: 1) **Algorithm Selection:** The root selects an Algo node with PUCT. 2) **Proposer Selection:** For the chosen algorithm, we estimate the GP surrogate’s quality to select the BO or LLM proposer. 3) **Proposal:** The BO proposer samples candidates and selects with EI; the LLM proposer extends PUCT to select a base node and prompts LLM to generate a new configuration. 4) **Playout & Reflection:** The configuration is evaluated, followed by a reflection, to add a new HP node. 5) **Backpropagation:** The result is propagated to update ancestral nodes’ statistics. After the budget is exhausted, LB-MCTS returns the configuration with the best validation performance.

Difference with Previous Methods. Unlike prior LLM optimizers for flat black-box problems, LB-MCTS targets structured CASH. Unlike existing LLM-BO hybrids that pass flat histories to LLMs, LB-MCTS unifies BO- and LLM-produced trajectories in a shared tree state, enabling richer path-aware reasoning. Unlike prior LLM-MCTS methods, LB-MCTS uses the MCTS tree as a shared CASH state, integrating algorithm selection and LLM-BO synergistic HPO into unified trajectories.

Theorem 3.1 (Global Convergence). *If the BO-selection probability is lower bounded by $\epsilon > 0$, LB-MCTS converges to the global optimum almost surely as the number of iterations $T \rightarrow \infty$:*

$$\lim_{T \rightarrow \infty} \left(\mathcal{F}(\lambda^*) - \max_{t=1}^T \mathcal{F}(\lambda_t) \right) = 0, \quad (7)$$

Appendix A.3 provides the detailed proof, including the finite-time regret bound in Theorem A.5 and the acceleration-ratio characterization in Corollary A.6, showing that LB-MCTS inherits BO’s convergence guarantee while leveraging LLMs for acceleration.

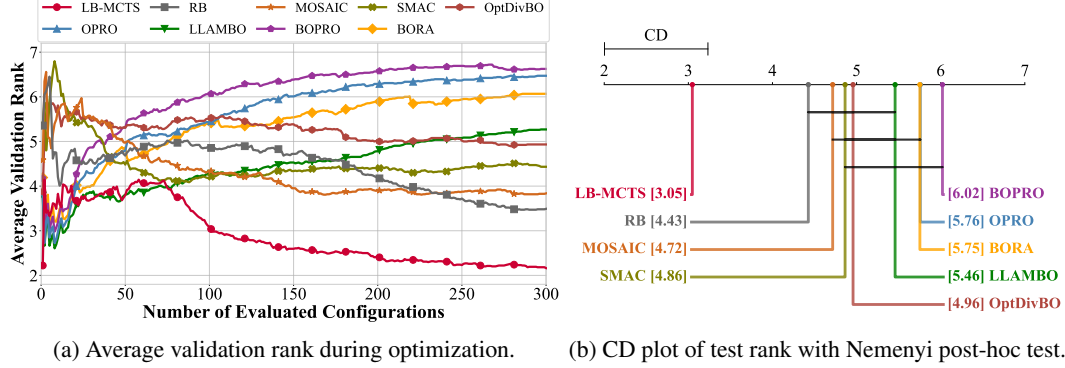


Figure 2: Average Performance of 9 methods across 104 datasets.

4 Experiment

4.1 Experiment Setup

Baselines. We compare LB-MCTS with 8 baselines: — *Four BO-based CASH methods*: 1) **SMAC** [15]: standard BO; 2) **OptDivBO** [33]: a BO variant encouraging diversity; 3) Rising Bandit (**RB**) [23]: a multi-armed-bandit extension of BO for CASH; 4) **MOSAIC**: a BO variant with PUCT for algorithm selection. — *Two pure LLM Optimizer*: 5) **OPRO**, — 6) **LLAMBO**, — *Two hybrid LLM-BO Methods*: 7) **BOPRO**, which optimizes LLM’s prompt instructions with BO; and 8) **BORA**, which switches between LLM and BO via a variance heuristic.

CASH space, datasets, and metrics. We construct a hierarchical CASH search space comprising 8 machine learning algorithms (e.g., XGBoost) for both classification and regression tasks, resulting in high-dimensional search spaces with 45 and 46 hyperparameters, respectively (see Appendix C.1). For datasets, we utilize the AutoML Benchmark (AMLB) [10], which contains 71 classification tasks and 33 regression tasks across varying scales and domains. Following [8], we report Balanced Accuracy for classification and Mean Squared Error (MSE) for regression as performance metrics.

Basic settings. Each dataset is split into training, validation, and test sets with a 60/20/20 ratio. We report the best-observed validation metric during optimization and the final test metric. While it takes a different amount of time to evaluate the same configuration on different datasets, we use the evaluation iterations as the unit of budget, where each baseline evaluates 300 configurations. We implement the BO surrogate, the random and local sampling mechanism using OpenBox 0.8.1 [17], an open-source toolkit. For the algorithm prior, the number of sampled configurations n_s is set to 100. The exploration constant in PUCT is set to $c_{puct} = \sqrt{2}$. We employ GPT-4o-mini [32], OpenAI’s cost-efficient model, as the backbone for all LLM-based optimizers (ablation in Appendix C.2). More implementation details for other baselines are provided in Appendix C.3.

4.2 Comparison with Baselines

This section evaluates LB-MCTS on 104 AMLB datasets. Figure 2a displays the average rank of the *best-achieved validation performance* at each iteration, from which we draw four observations: (i) Current LLM-based baselines (BORA, LLAMBO, BOPRO, and OPRO) perform poorly on CASH. They are competitive only in the first ~ 50 iterations; afterwards their average ranks steadily deteriorate, ending between 5.27 and 6.63—worse than other methods. This is because they treat CASH as a flat joint space, mix histories across algorithms, and lack explicit exploration-exploitation control, getting stuck in local optima in the structured, high-dimensional search space. (ii) Among LLM-based optimizers, OPRO and BOPRO perform worst, as they rely on simplistic iterative prompting where BO (in BOPRO’s case) is limited to superficial example selection. BORA (rank 6.07) gains slightly via LLM–BO switching, but its behavior is governed by fixed uncertainty thresholds that do not transfer well to CASH. LLAMBO attains the highest rank (5.27) but incurs prohibitive computational overhead. (iii) Among BO-based baselines, OptDivBO performs worst (rank 4.93) as it prioritizes ensemble diversity over individual model performance. MOSAIC improves on SMAC by decoupling algorithm selection from HPO. RB leads BO baselines by using local surrogates after algorithm selection instead of a global surrogate over the joint space. (iv) Among all methods,

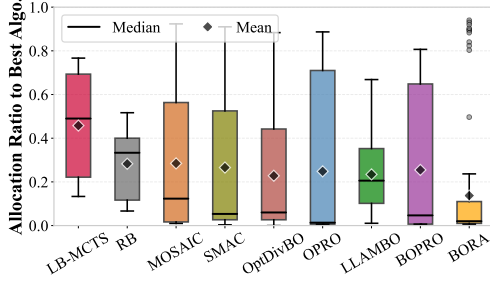


Figure 3: Resource allocation.

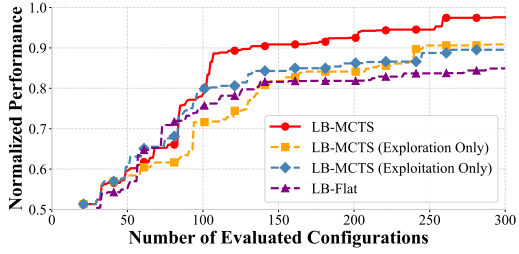


Figure 4: Impact of the tree-structured search.

LB-MCTS significantly dominates the others. It consistently outperforms all baselines beyond around 70 iterations. While the second-best baseline (RB) ranks 3.49 finally, **the rank of LB-MCTS is 2.16**. Appendix C.4 further analyzes these results by task characteristics, showing that LB-MCTS remains robust across diverse tasks. To further assess generality beyond conventional ML algorithms, Appendix C.5 reports supplementary experiments on a broader search space including deep learning models and feature engineering, where LB-MCTS continues to achieve the best average rank.

Test set performance. We further *evaluate the validation-best configurations on the test set* and illustrate the results in the Critical Difference (CD) diagram (Figure 2b). We observe that the ranks of methods on the test set may deviate from their validation ranks. The reason is that the distributions of the validation and test set are not exactly the same [16]. But overall, LB-MCTS remains statistically significantly superior to all baselines, attaining a **leading average rank of 3.05**. To further assess the practical utility, we evaluate the post-hoc ensemble performance of all methods in Appendix C.6.

The success of LB-MCTS stems from two main factors. First, LB-MCTS remedies key limitations of prior LLM-based optimizers: (i) The STM mechanism isolates algorithm-specific histories and combines global and local memory to enhance reasoning (validated in Appendix C.12). (ii) As shown in Section 4.3, MCTS enables a principled exploration–exploitation trade-off both across algorithms and within each algorithm’s subtree. (iii) The reflection mechanism further strengthens STM by turning numeric feedback into reusable verbal guidance (Appendix C.11). Second, LB-MCTS achieves optimal synergy between LLM and BO proposers, outperforming either component alone (Section 4.4). Appendix C.7 provides analyses of both API cost and wall-clock overhead, showing that LB-MCTS achieves these gains with an economical cost of about \$0.127 per task.

4.3 MCTS for Exploration-Exploitation Trade-off

In this section, we validate the ability of LB-MCTS to trade off exploration and exploitation at two levels: (1) Can it allocate optimization resources to superior algorithms? (2) Within each algorithm’s subspace, can it effectively navigate the trade-off between promising and unexplored regions?

Resource allocation across algorithms. For each dataset, we first identified the optimal algorithm as the algorithm associated with the global best configuration found across all 9 methods and 300 iterations. Subsequently, for each CASH method, we compute the *ratio of its evaluated configurations that belong to the “optimal algorithm”*. Figure 3 visualizes the distribution of these allocation ratios for each method across the 104 datasets, from which we get two observations: (i) **Superiority**: LB-MCTS achieves the highest mean allocation ratios (0.43), whereas other baselines allocate 0.14 to 0.28 on average. This demonstrates its superior ability to correctly identify the most promising algorithm and concentrate optimization budget on it. (ii) **Robustness**: LB-MCTS exhibits the highest median (0.49), along with highest lower quartile (box bottom) and lower whisker, ensuring the optimal algorithm is rarely overlooked. In contrast, aggressive baselines (e.g., SMAC, OPRO, MOSAIC) exhibit near-zero medians and bottom whiskers, indicating frequent premature convergence to suboptimal algorithms; methods like RB and LLAMBO yield consistently low ratios due to conservative uniformity. In summary, LB-MCTS employs root-level PUCT to balance inter-algorithm trade-off, ensuring stable resource allocation to optimal algorithms across datasets.

Navigating within each algorithm’s subspace. Effective HPO requires balancing exploiting promising regions and exploring unknown areas to avoid local optima. To achieve this, LB-MCTS explicitly prompts the LLM proposer to *expand the tree structure using two directives*: EXPLOITATION for local refinement and EXPLORATION for global search. We compare this design against two single directive

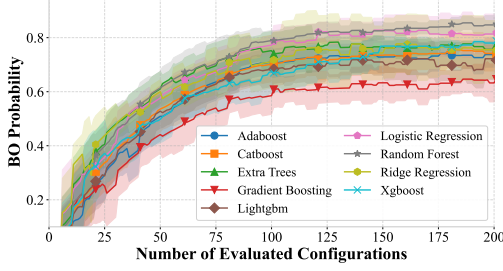


Figure 5: Average P_{BO} for each algorithm.

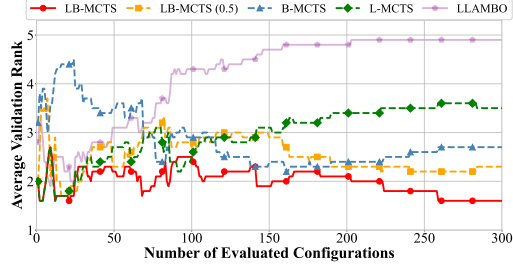


Figure 6: Average rank during optimization.

variants—EXPLORATION only and EXPLOITATION only—expanded twice per node, and LB-Flat, which retains only the root and Algo Nodes, substituting subsequent tree search with unstructured history sharing between BO and LLM (an OPRO-style proposer). To conserve API costs, all ablation studies are conducted on 10 representative datasets from AMLB (details in Appendix C.8). Figure 4 plots the average *best-so-far validation performance* across the 10 datasets, where scores on each dataset are min-max normalized globally. The results show that relying solely on EXPLOITATION yields rapid initial gains but leads to premature stagnation, trapped in local optima. Conversely, using only EXPLORATION results in slower convergence due to insufficient refinement of promising regions. LB-MCTS outperforms both variants and LB-Flat, finally achieving **an average rank of 1.5**, compared to 2.3 and 2.4 for the two variants. Most importantly, the poor performance of LB-Flat (rank 3.4) underscores that unstructured history sharing—a common trait in existing methods—is insufficient for complex CASH problems without the hierarchical guidance of MCTS. Details of LB-Flat and further analysis are provided in Appendix C.9. In summary, the synergy of dual directives and tree-structured search enables LB-MCTS to effectively navigate the search space to avoid the local optima. Additional ablations in Appendix C.10 validate the choice of PUCT over alternative tree-search strategies and show that $c_{puct} = \sqrt{2}$ provides robust exploration.

4.4 Analysis of BO-LLM Collaboration

LB-MCTS maintains a BO-selection probability (P_{BO}) for each algorithm and samples the proposer at every HPO step. Figure 5 plots the average P_{BO} for each algorithm versus the number of evaluations over 104 datasets. Initially ($N < 25$), P_{BO} is low (< 0.3), reflecting reliance on semantic priors when data are scarce. As the number of evaluations grows, the BO surrogate becomes more reliable: around ~ 50 evaluations, P_{BO} rises to ≈ 0.5 , marking a transition. After ~ 100 evaluations, it gradually stabilizes between 0.6 and 0.8, with modest variation across algorithms (likely reflects differences in landscape smoothness). On average, the LLM proposer is selected in 107 out of 300 iterations.

To quantify the benefit of this dynamic synergy, we perform an ablation study on 10 datasets against three variants: L-MCTS (pure LLM), B-MCTS (pure BO), and LB-MCTS(0.5) (fixed 50:50 probability). We also include LLAMBO, the strongest LLM-based baseline. Figure 6 tracks the average rank of the best validation performance. (i) L-MCTS dominates early (< 70 iterations), while B-MCTS prevails later (> 110 iterations), confirming LLMs excel at warm-starting while BO excels with sufficient data. (ii) Benefit of synergy: both LB-MCTS (0.5) and LB-MCTS outperform single-proposer variants. (iii) LB-MCTS achieves the best performance over the entire optimization, achieving a final **rank of 1.6** versus 2.3 for LB-MCTS (0.5), confirming the value of dynamic synergy. Notably, LB-MCTS outperforms L-MCTS even in the early phase, suggesting that occasional injections from the non-dominant proposer provide necessary diversity for performance gains. (iv) Finally, L-MCTS (rank 3.5) significantly outperforms LLAMBO (rank 4.9), reaffirming that our trajectory-aware MCTS enables more effective LLM reasoning than unstructured prompting.

5 Conclusion

In this paper, we propose LB-MCTS, which synergizes LLMs and BO via tree search for CASH. LB-MCTS introduces three key components: MCTS to balance exploration and exploitation hierarchically, Selective Tuning Memory to enhance LLM reasoning, and a dynamic mechanism to select between LLM and BO proposers. Experiments on 104 AMLB datasets show that LB-MCTS outperforms BO-based and LLM-based baselines. We discuss limitations and future directions in Appendix A.1.

References

- [1] Dhruv Agarwal, Manoj Ghuhana Arivazhagan, Rajarshi Das, Sandesh Swamy, Sopan Khosla, and Rashmi Gangadharaiah. Searching for optimal solutions with llms via bayesian optimization. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [2] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyperparameter optimization. *Advances in neural information processing systems*, 24, 2011.
- [3] Guillaume Chaslot, Sander Bakkes, Istvan Szita, and Pieter Spronck. Monte-carlo tree search: A new framework for game ai. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 4, pages 216–217, 2008.
- [4] Abdoulatif Cissé, Xenophon Evangelopoulos, Vladimir V Gusev, and Andrew I Cooper. Language-based bayesian optimization research assistant (bora). *arXiv preprint arXiv:2501.16224*, 2025.
- [5] Gustavo Sutter Pessurno de Carvalho, Mohammed Abdulrahman, Hao Wang, Sriram Ganapathi Subramanian, Marc St-Aubin, Sharon O’Sullivan, Lawrence Wan, Luis Ricardez-Sandoval, Pascal Poupart, and Agustinus Kristiadi. Simplifying bayesian optimization via in-context direct optimum sampling. *arXiv preprint arXiv:2505.23913*, 2025.
- [6] Iddo Drori, Yamuna Krishnamurthy, Remi Rampin, Raoni DE PAULA LOURENCO, Jorge Piazzentin Ono, Kyunghyun Cho, Claudio Silva, and Juliana Freire. Alphad3m: Machine learning pipeline synthesis. In *ICML AutoML Workshop*, 2021.
- [7] Stefan Falkner, Aaron Klein, and Frank Hutter. Bohb: Robust and efficient hyperparameter optimization at scale. In *International conference on machine learning*, pages 1437–1446. PMLR, 2018.
- [8] Matthias Feurer, Katharina Eggenberger, Stefan Falkner, Marius Lindauer, and Frank Hutter. Auto-sklearn 2.0: Hands-free automl via meta-learning. *Journal of Machine Learning Research*, 23(261):1–61, 2022.
- [9] Sylvain Gelly and David Silver. Monte-carlo tree search and rapid action value estimation in computer go. *Artificial Intelligence*, 175(11):1856–1875, 2011.
- [10] Pieter Gijsbers, Erin LeDell, Janek Thomas, Sébastien Poirier, Bernd Bischl, and Joaquin Vanschoren. An open source automl benchmark. *arXiv preprint arXiv:1907.00909*, 2019.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [12] Xin He, Kaiyong Zhao, and Xiaowen Chu. Automl: A survey of the state-of-the-art. *Knowledge-based systems*, 212:106622, 2021.
- [13] Noah Hollmann, Samuel Müller, and Frank Hutter. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *Advances in Neural Information Processing Systems*, 36:44753–44775, 2023.
- [14] Beichen Huang, Xingyu Wu, Yu Zhou, Jibin Wu, Liang Feng, Ran Cheng, and Kay Chen Tan. Exploring the true potential: Evaluating the black-box optimization capability of large language models. *arXiv preprint arXiv:2404.06290*, 2024.
- [15] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *International Conference on Learning and Intelligent Optimization*, pages 507–523. Springer, 2011.
- [16] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- [17] Huaijun Jiang, Yu Shen, Yang Li, Beicheng Xu, Sixian Du, Wentao Zhang, Ce Zhang, and Bin Cui. Openbox: A python toolkit for generalized black-box optimization. *Journal of Machine Learning Research*, 25(120):1–11, 2024.

- [18] Donald R Jones, Matthias Schonlau, and William J Welch. Efficient global optimization of expensive black-box functions. *Journal of Global optimization*, 13(4):455–492, 1998.
- [19] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.
- [20] Nicholas Kroeger, Dan Ley, Satyapriya Krishna, Chirag Agarwal, and Himabindu Lakkaraju. In-context explainers: Harnessing llms for explaining black box models. *arXiv preprint arXiv:2310.05797*, 2023.
- [21] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
- [22] Erin LeDell and Sebastien Poirier. H2o automl: Scalable automatic machine learning. In *Proceedings of the AutoML Workshop at ICML*, volume 2020, page 24, 2020.
- [23] Yang Li, Jiawei Jiang, Jinyang Gao, Yingxia Shao, Ce Zhang, and Bin Cui. Efficient automatic cash via rising bandits. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 4763–4771, 2020.
- [24] Sijia Liu, Parikshit Ram, Deepak Vijaykeerthy, Djallel Bouneffouf, Gregory Bramble, Horst Samulowitz, Dakuo Wang, Andrew Conn, and Alexander Gray. An admm based framework for automl pipeline configuration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 4892–4899, 2020.
- [25] Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. Large language models to enhance bayesian optimization. In *The Twelfth International Conference on Learning Representations*, 2025.
- [26] Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, Yanfeng Wang, Siheng Chen, et al. ML-master: Towards ai-for-ai via integration of exploration and reasoning. *arXiv preprint arXiv:2506.16499*, 2025.
- [27] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024.
- [28] Kanan Mahammadli and Seyda Ertekin. Sequential large language model-based hyper-parameter optimization. *arXiv preprint arXiv:2410.20302*, 2024.
- [29] Nicolas Menet, Aleksandar Terzić, Michael Hersche, Andreas Krause, and Abbas Rahimi. Thompson sampling via fine-tuning of llms. *arXiv preprint arXiv:2510.13328*, 2025.
- [30] Tung Nguyen and Aditya Grover. Lico: Large language models for in-context molecular optimization. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [31] Randal S Olson and Jason H Moore. Tpot: A tree-based pipeline optimization tool for automating machine learning. In *Workshop on automatic machine learning*, pages 66–74. PMLR, 2016.
- [32] OpenAI. Gpt-4o-mini model card., 2024. URL <https://platform.openai.com/docs/models/gpt-4o-mini>.
- [33] Pranav Poduval, Sanjay Kumar Patnala, Gaurav Oberoi, Nitish Srivasatava, and Siddhartha Asthana. Cash via optimal diversity for ensemble learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2411–2419, 2024.
- [34] Herilalaina Rakotoarison, Marc Schoenauer, and Michèle Sebag. Automated machine learning with monte-carlo tree search. In *IJCAI-19-28th International Joint Conference on Artificial Intelligence*, pages 3296–3303. International Joint Conferences on Artificial Intelligence Organization, 2019.
- [35] Bojana Ranković and Philippe Schwaller. Bochemian: Large language model embeddings for bayesian optimization of chemical reactions. In *NeurIPS 2023 Workshop on Adaptive Experimental Design and Active Learning in the Real World*, 2023.

- [36] Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.
- [37] Yu Shen, Yupeng Lu, Yang Li, Yaofeng Tu, Wentao Zhang, and Bin Cui. Divbo: diversity-aware cash for ensemble learning. *Advances in Neural Information Processing Systems*, 35:2958–2971, 2022.
- [38] Z Shi, M Fang, and L Chen. Monte carlo planning with large language model for text-based game agents. In *13th International Conference on Learning Representations Iclr 2025*, 2025.
- [39] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- [40] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.
- [41] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in neural information processing systems*, volume 25, pages 2951–2959, 2012.
- [42] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1441–1450, 2019.
- [43] Chris Thornton, Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Auto-weka: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 847–855, 2013.
- [44] Anton Vakhrushev, Alexander Ryzhkov, Maxim Savchenko, Dmitry Simakov, Rinchin Damdinov, and Alexander Tuzhilin. Lightautoml: Automl solution for a large financial services ecosystem. *arXiv preprint arXiv:2109.01528*, 2021.
- [45] Niki van Stein, Diederick Vermetten, and Thomas Bäck. In-the-loop hyper-parameter optimization for llm-based automated design of heuristics. *ACM Transactions on Evolutionary Learning*, 2024.
- [46] Gang Wang, Xiang Li, Zi-Yi Guo, Da-Wei Yin, and Shuai Ma. Smec: Scene mining for e-commerce. *Journal of Computer Science and Technology*, 39(1):192–210, 2024.
- [47] Christopher KI Williams and Carl Edward Rasmussen. *Gaussian processes for machine learning*, volume 2. MIT press Cambridge, MA, 2006.
- [48] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2023.
- [49] Yuxuan Yin, Yu Wang, Boxun Xu, and Peng Li. Ado-llm: Analog design bayesian optimization with in-context learning of large language models. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2024.
- [50] Zirui Zhao, Wee Sun Lee, and David Hsu. Large language models as commonsense knowledge for large-scale task planning. *Advances in neural information processing systems*, 36:31967–31987, 2023.

A Method Details

A.1 Limitation and Future Work

Our method inherits the general challenge that validation feedback can be noisy in some scenarios, such as when the validation set is small or the data itself is highly noisy. In these cases, the resulting feedback signal may be less reliable and can occasionally mislead MCTS and the LLM proposer. While BO has built-in mechanisms to better tolerate noisy evaluations, the current LLM proposer does not explicitly model such noise. A promising direction for future work is therefore to develop noise-aware LLM tuning strategies, such as prompt designs or feedback processing schemes that are more robust to uncertain validation signals.

In addition, the effectiveness of our framework still depends to some extent on the capability of the underlying LLM. Although our experiments show that several relatively economical models, such as GPT-4o-mini and DeepSeek-V3, already perform well in our setting, weaker models may nevertheless provide less informative proposals and thus limit the overall gain. Future improvements could come from lightweight domain adaptation, fine-tuning, or the incorporation of external tools to further strengthen the tuning ability of smaller or less capable models.

From a broader-impact perspective, LB-MCTS may lower the expertise and cost barriers for applying AutoML by improving the efficiency of CASH optimization. However, as with other AutoML systems, the selected models may still inherit biases, privacy risks, or reliability issues from the underlying data and downstream application context. Therefore, when applying LB-MCTS to sensitive domains, users should conduct appropriate data governance, fairness checks, and domain-specific validation before deployment.

A.2 Pseudocode of LB-MCTS

Algorithm 1 summarizes the procedure of LB-MCTS. The detailed workflow is as follows.

Initialization (Lines 2–5). Given a dataset $\mathcal{D}$ and a set of candidate algorithms $\mathcal{A} = \{A^1, \dots, A^K\}$ with corresponding hyperparameter spaces $\{\Lambda^i\}_{i=1}^K$, we construct the search tree with a root CASH node s_{root} and K Algo children $\{s_{\text{algo}}^i\}_{i=1}^K$ (Lines 2–3). For each algorithm A^i , we maintain: (i) a local dataset $\mathcal{D}_i$ of evaluated configurations, (ii) a GP surrogate f_i over Λ^i , (iii) an episode set $\mathcal{E}_i$ storing optimization attempts (parent/child configurations, performances, and reflections).

While the budget T is not exhausted, LB-MCTS execute an MCTS loop over the CASH tree:

Algorithm selection at the root (Lines 7–9); Section 3.2. At each iteration, LB-MCTS first decides which algorithm to optimize by applying the PUCT rule (Equation (2)) to select an Algo node s_{algo}^i .

Dynamic proposer selection (Lines 10–15); Section 3.3.3. Given the chosen algorithm A^i , we decide whether the next configuration should be proposed by BO or LLM. We estimate the generalization quality of the surrogate f_i on the current local dataset via cross-validation, compute the BO selection probability, and sample $z \sim \text{Bernoulli}(p_{\text{BO}}^i)$. When the surrogate is still inaccurate (small τ^i), the LLM proposer is chosen more often; as more data accumulate and the surrogate improves, BO is selected with higher probability. Specifically, p_{BO}^i is initialized to ϵ and updated independently for each algorithm A^i **whenever five new configurations have been added to its local history**.

Proposing a new configuration. (Lines 16–27) The configuration proposal depends on whether $z = 1$ (BO) or 0 (LLM).

- **BO proposer** (Line 17–19); Section 3.3.1. When $z = 1$, we call BOPROPOSE with the current algorithm A^i , its Algo node s_{algo}^i , the surrogate f_i , and data $\mathcal{D}_i$. This routine: 1) constructs a candidate pool by mixing *random search* candidates drawn uniformly from Λ^i (initialization actions), and *local search* candidates sampled in the neighborhoods of previously evaluated configurations; 2) scores all candidates with an acquisition function (we use EI under the GP f_i); 3) selects the candidate λ_{new} with the highest EI and identifies its parent node s_{parent} in the tree (the Algo node for random samples or the corresponding HP node for local-search samples).
- **LLM proposer** (Lines 20–26); Section 3.3.2. When $z = 0$, we use the LLM proposer. 1) First, we extend the MCTS Selection phase into the algorithm subtree by applying HP-level PUCT (Line 22). This yields a base HP node s_{base} and its configuration λ_{base} . 2) Next, we build the

Algorithm 1 LB-MCTS: LB-MCTS for CASH

```
1: Input: dataset  $\mathcal{D}$ , algorithms  $\mathcal{A} = \{A^1, \dots, A^K\}$  with spaces  $\{\Lambda^i\}$ , evaluation budget  $T$ 
2: Initialize root node  $s_{\text{root}}$  with Algo children  $\{s_{\text{algo}}^i\}_{i=1}^K$ 
3: Initialize per-algorithm data  $\mathcal{D}_i \leftarrow \emptyset$ , surrogate  $f_i$  (GP), episode set  $\mathcal{E}_i \leftarrow \emptyset$ ,  $p_{\text{BO}}^i \leftarrow \epsilon$ 
4: Initialize node stats  $N_s \leftarrow 0$ ,  $R_s \leftarrow 0$ ,  $y_s^{\text{max}} \leftarrow -\infty$  for all  $s$ 
5:  $t \leftarrow 0$ 
6: while  $t < T$  do
7:   // Algorithm selection (root PUCT)
8:   Compute  $Q(s_{\text{algo}}^i)$  and prior  $P(s_{\text{algo}}^i)$  for all  $i$  (Sec. 3.2)
9:   Select algorithm node  $s_{\text{algo}}^i$  by PUCT (Eq. 2); let  $A^i$  be the chosen algorithm
10:  // Dynamic proposer selection
11:  if  $|\mathcal{D}_i| > 0$  and  $|\mathcal{D}_i| \pmod{5} = 0$  then
12:    Estimate surrogate quality  $\tau^i$  via  $k$ -fold CV on  $\mathcal{D}_i$ 
13:     $p_{\text{BO}}^i \leftarrow \max(\epsilon, \frac{\tau^i + 1}{2})$ 
14:  end if
15:  Sample  $z \sim \text{Bernoulli}(p_{\text{BO}}^i)$ 
16:  // HPO with LLM/BO
17:  if  $z = 1$  then
18:    {use BO proposer}
19:     $(\lambda_{\text{new}}, s_{\text{parent}}) \leftarrow \text{BOPROPOSE}(A^i, s_{\text{algo}}^i, f_i, \mathcal{D}_i)$ 
20:  else
21:    {use LLM proposer}
22:     $(\lambda_{\text{base}}, s_{\text{base}}) \leftarrow \text{PUCTSELECTHP}(s_{\text{algo}}^i)$  {HP-level PUCT with uniform prior}
23:    Build STM  $\mathcal{M}_i(s_{\text{base}})$  from global episodes  $\mathcal{E}_i$  and ancestors of  $s_{\text{base}}$ 
24:    Choose directive  $d \in \{\text{WARMUP}, \text{EXPLORATION}, \text{EXPLOITATION}\}$  for  $s_{\text{base}}$ 
25:     $(\mathcal{R}, \lambda_{\text{new}}) \leftarrow \text{LLM}(\psi, \mathcal{M}_i(s_{\text{base}}), \lambda_{\text{base}}, d)$ 
26:     $s_{\text{parent}} \leftarrow s_{\text{base}}$ 
27:  end if
28:  // Payout and reflection
29:  Evaluate  $A^i$  with  $\lambda_{\text{new}}$  on  $\mathcal{D}$  to obtain  $y_{\text{new}}$ 
30:  Let  $(\lambda_{\text{base}}, y_{\text{base}})$  be the config/performance of  $s_{\text{parent}}$  (if any)
31:  if  $z = 1$  then
32:    {BO: heuristic reflection}
33:     $\rho_{\text{new}} \leftarrow \text{SYNTHESIZESUMMARY}(\Delta\lambda, \Delta y)$ 
34:  else
35:    {LLM proposer: verbal reflection}
36:     $\rho_{\text{new}} \leftarrow \text{LLM}(\psi, \mathcal{M}_i(s_{\text{parent}}), \lambda_{\text{base}}, y_{\text{base}}, \lambda_{\text{new}}, y_{\text{new}})$ 
37:  end if
38:  Create new HP node  $s_{\text{new}}$  with  $(\lambda_{\text{new}}, y_{\text{new}}, \rho_{\text{new}})$  as child of  $s_{\text{parent}}$ 
39:  Add  $(\lambda_{\text{base}}, y_{\text{base}}, \lambda_{\text{new}}, y_{\text{new}}, \rho_{\text{new}})$  to episode set  $\mathcal{E}_i$ 
40:  Update dataset  $\mathcal{D}_i \leftarrow \mathcal{D}_i \cup \{(\lambda_{\text{new}}, y_{\text{new}})\}$  and refit GP  $f_i$  (periodically)
41:  // Backpropagation
42:  for each node  $s$  on path from  $s_{\text{root}}$  to  $s_{\text{new}}$  do
43:     $r_s \leftarrow [y_{\text{new}} - y_s^{\text{max}}]_+ / |y_s^{\text{max}}|$ 
44:     $N_s \leftarrow N_s + 1$ ;  $R_s \leftarrow R_s + r_s$ ;  $y_s^{\text{max}} \leftarrow \max(y_s^{\text{max}}, y_{\text{new}})$ 
45:  end for
46:   $t \leftarrow t + 1$ 
47: end while
48: Output: best  $(A^*, \lambda^*)$  observed so far (maximizing validation performance)
```

Selective Tuning Memory $\mathcal{M}_i(s_{\text{base}})$ (Line 23), which combines: a *global memory* of Pareto-selected episodes and a *local memory* consisting of all HP nodes along the ancestral path from s_{algo}^i to s_{base} . 3) Then we choose a directive $d \in \{\text{WARMUP}, \text{EXPLORATION}, \text{EXPLOITATION}\}$ according to the expansion state of s_{base} (Line 24). 4) We then query the LLM to obtain a

reasoning chain $\mathcal{R}$ and a proposed configuration λ_{new} (Line 25). In this case, the parent node is $s_{\text{parent}} = s_{\text{base}}$ (Line 26).

Playout and reflection (Lines 28–37). As described in Section 3.3.2, the newly proposed configuration λ_{new} is evaluated on $\mathcal{D}$ to obtain the validation performance y_{new} (Line 29). We then attach a reflection ρ_{new} to this trial, with different generation mechanisms depending on the proposer:

- **BO proposer** (Lines 32–33). We call a SYNTHESIZESUMMARY to generate a heuristic textual reflection from the parameter change $\Delta\lambda$ and performance change Δy (Line 33). This yields short descriptions such as “*Increasing depth from 4 to 6 improved accuracy by 1.2%*”.
- **LLM proposer** (Lines 35–36). We call the LLM to produce a semantic reflection, which explains why the modification helped or hurt and suggests how to adjust future configurations.

We then create a new HP node s_{new} as a child of s_{parent} , storing $(\lambda_{\text{new}}, y_{\text{new}}, \rho_{\text{new}})$. The episode $(\lambda_{\text{base}}, y_{\text{base}}, \lambda_{\text{new}}, y_{\text{new}}, \rho_{\text{new}})$ is added to the episode set $\mathcal{E}_i$, and the local dataset $\mathcal{D}_i$ is augmented with the new pair $(\lambda_{\text{new}}, y_{\text{new}})$; the surrogate f_i is periodically refit on $\mathcal{D}_i$ (Lines 38–40).

Backpropagation (Lines 41–46). Finally, we propagate the result back through the tree to update node statistics.

Output (Line 49) After the evaluation budget T is exhausted, LB-MCTS returns the best combination (A^i, λ) observed so far in terms of validation performance, providing a CASH solution obtained through the coordinated action of MCTS-guided selection, BO-based quantitative search, and LLM-based qualitative reasoning.

A.3 Finite-Time Regret Analysis

We provide a finite-time version of Theorem 3.1. Let A^* denote an algorithm branch that contains a global maximizer, and let $N_*(T)$ be the number of visits to this branch within a total budget T . Among these visits, let $M_*(T)$ be the number of times the BO proposer is selected. The global simple regret is

$$S_T^{\text{Global}} = \mathcal{F}(\lambda^*) - \max_{1 \leq t \leq T} \mathcal{F}(\lambda_t).$$

Assumption A.1 (Finite-time root allocation). Let $*$ be the index of the optimal branch A^* . Let $N_i(t)$ be the number of visits to algorithm branch A^i after t root selections, and denote its root prior by $P_i = P(s_{\text{algo}}^i)$. Assume that, after initial expansion, the root-level value estimates are separated by gaps $\Delta_i > 0$:

$$Q_t(A^*) - Q_t(A^i) \geq \Delta_i, \quad \forall i \neq *, \quad \forall t \leq T.$$

Under the PUCT selection rule in Eq. (2), if a suboptimal branch A^i is selected at round t , then

$$Q_t(A^i) + c_{\text{puct}} P_i \frac{\sqrt{t}}{1 + N_i(t)} \geq Q_t(A^*) + c_{\text{puct}} P_* \frac{\sqrt{t}}{1 + N_*(t)} \geq Q_t(A^*).$$

Combining the gap condition and the PUCT selection inequality gives

$$1 + N_i(t) \leq \frac{c_{\text{puct}} P_i \sqrt{t}}{\Delta_i} \leq \frac{c_{\text{puct}} P_i \sqrt{T}}{\Delta_i}.$$

Thus each suboptimal branch is visited at most $\mathcal{O}(\sqrt{T}/\Delta_i)$ times, and the number of visits allocated to A^* satisfies

$$N_*(T) \geq T - \sum_{i \neq *} \frac{c_{\text{puct}} P_i \sqrt{T}}{\Delta_i} \geq T - \sum_{i \neq *} \frac{c_{\text{puct}} \sqrt{T}}{\Delta_i} = T - \mathcal{O}(K\sqrt{T}).$$

Thus $N_*(T) = \Omega(T)$ as $T \rightarrow \infty$. This condition isolates the well-separated root-selection regime; the finite-time regret analysis below then focuses on proposer collaboration within the optimal branch.

Assumption A.2 (Finite-time BO regret). Conditioned on m BO evaluations and q evaluated configurations proposed by the LLM in the optimal branch, denoted by $\mathcal{D}_q^{\text{LLM}}$, the EI proposer is warm-started with this dataset and satisfies the branch-wise simple-regret rate of expected improvement [54]:

$$S_m^{\text{BO|LLM}} \leq C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}}) r_{\text{EI}}(m),$$

where $C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}}) > 0$ is a data-dependent constant induced by the warm-start observations and

$$r_{\text{EI}}(m) = \begin{cases} m^{-\nu/d_*} (\log m)^\eta, & 0 < \nu \leq 1, \\ m^{-1/d_*}, & \nu > 1. \end{cases} \quad (8)$$

Here d_* is the dimension of the optimal branch, ν is the RKHS smoothness parameter of the GP kernel, and $\eta \geq 0$ denotes the logarithmic factor in the EI convergence rate. High-quality LLM proposals can reduce the finite-time constant by lowering the initial simple regret and providing informative posterior observations, while the EI exponent $r_{\text{EI}}(m)$ remains unchanged.

Assumption A.3 (LLM-assisted proposal quality). Motivated by empirical in-context scaling laws for LLMs [51, 61], we model the LLM-assisted regret after n observations in the optimal branch as a power-law decay with an irreducible floor:

$$S_n^{\text{LLM}} \leq \frac{C_{\text{LLM}}}{n^\alpha} + \Delta_{\text{inf}}, \quad (9)$$

where $C_{\text{LLM}} > 0$, $\alpha > 0$, and $\Delta_{\text{inf}} \geq 0$ is the irreducible floor of the LLM proposer. This assumption formalizes the finite-time benefit of semantic priors.

Lemma A.4 (BO evaluations in the optimal branch). *Suppose that whenever A^* is selected, the BO proposer is selected with conditional probability at least $\epsilon > 0$. Then*

$$\Pr\left(M_*(T) \geq \frac{\epsilon N_*(T)}{2} \mid N_*(T)\right) \geq 1 - \exp\left(-\frac{\epsilon N_*(T)}{8}\right).$$

Proof. Index the visits to A^* by $j = 1, \dots, N_*(T)$. The proposer-selection variable can be written as $Z_j = \mathbb{I}\{U_j \leq p_j\}$, where U_j are independent uniform random variables and the adaptive probability p_j satisfies $p_j \geq \epsilon$. Thus Z_j stochastically dominates $\mathbb{I}\{U_j \leq \epsilon\}$, and $M_*(T)$ dominates a binomial random variable with parameters $N_*(T)$ and ϵ . The multiplicative Chernoff bound gives

$$\Pr\left(M_*(T) < \frac{\epsilon N_*(T)}{2}\right) \leq \exp\left(-\frac{\epsilon N_*(T)}{8}\right),$$

which proves the claim. $\square$

Theorem A.5 (Finite-time global simple regret). *Under Assumptions A.1–A.3, with probability at least*

$$1 - \exp\left(-\frac{\epsilon N_*(T)}{8}\right),$$

the global simple regret of LB-MCTS satisfies

$$S_T^{\text{Global}} \leq \min \left\{ \frac{C_{\text{LLM}}}{N_*(T)^\alpha} + \Delta_{\text{inf}}, C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}}) r_{\text{EI}}(M_*(T)) \right\}. \quad (10)$$

Proof. By Assumption A.1, $N_*(T) = \Omega(T)$. Conditioned on this allocation, Lemma A.4 implies that the event $\mathcal{E}_{\text{count}} = \{M_*(T) \geq \epsilon N_*(T)/2\}$ holds with probability at least $1 - \exp(-\epsilon N_*(T)/8)$.

On $\mathcal{E}_{\text{count}}$, Assumption A.2 gives

$$S_{M_*(T)}^{\text{BO|LLM}} \leq C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}}) r_{\text{EI}}(M_*(T)).$$

Assumption A.3 gives

$$S_{N_*(T)}^{\text{LLM}} \leq \frac{C_{\text{LLM}}}{N_*(T)^\alpha} + \Delta_{\text{inf}}.$$

The best configuration returned by LB-MCTS is the best point observed over the entire global history. Therefore, its global simple regret is no larger than the regret achieved by either the BO-generated points or the LLM-assisted points inside the optimal branch:

$$S_T^{\text{Global}} \leq \min \left\{ S_{N_*(T)}^{\text{LLM}}, S_{M_*(T)}^{\text{BO|LLM}} \right\}.$$

Combining the two branch-wise bounds with the root-allocation and proposer-count events yields Eq. (10). $\square$

Proof of Theorem 3.1. By Assumption A.1, $N_*(T) = \Omega(T)$ as $T \rightarrow \infty$. Since $N_*(T) = \Omega(T)$, the failure probabilities $\exp(-\epsilon N_*(T)/8)$ are summable. By Lemma A.4 and the Borel–Cantelli lemma, the event $M_*(T) \geq \epsilon N_*(T)/2$ fails only finitely often almost surely. Hence $M_*(T) \rightarrow \infty$ almost surely. The EI rate satisfies $r_{\text{EI}}(M_*(T)) \rightarrow 0$. Assumption A.2 gives the warm-started BO bound

$$S_{M_*(T)}^{\text{BO|LLM}} \leq C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}})r_{\text{EI}}(M_*(T)).$$

The warm-start set affects only the data-dependent constant; along the optimization trajectory this constant is finite and remains bounded, so the BO term converges to zero. Therefore, the BO term in Eq. (10) converges to zero almost surely, and because the global simple regret is upper-bounded by the minimum in Eq. (10), the global simple regret also converges to zero almost surely. $\square$

Corollary A.6 (Finite-time acceleration in the experimental setting). *In our experimental setting, the Matérn kernel uses smoothness $\nu > 1$. Therefore, the EI rate in Eq. (8) becomes*

$$r_{\text{EI}}(m) = m^{-1/d_*}.$$

For a target simple regret δ , vanilla EI in the original space requires, up to logarithmic factors,

$$T_{\text{Base}}(\delta) = \tilde{\mathcal{O}}\left(\left(\frac{C_{\text{EI}}}{\delta}\right)^{d_*}\right)$$

evaluations. By Eq. (10), LB-MCTS reaches the same target once either the LLM term or the BO term falls below δ . For $\delta > \Delta_{\text{inf}}$, this gives

$$T_{\text{LB}}(\delta) = \tilde{\mathcal{O}}\left(\min\left\{\left(\frac{C_{\text{LLM}}}{\delta - \Delta_{\text{inf}}}\right)^{1/\alpha}, \frac{1}{\epsilon}\left(\frac{C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}})}{\delta}\right)^{d_*}\right\}\right),$$

where lower-order root-allocation constants are omitted for readability. When $\delta \leq \Delta_{\text{inf}}$, the LLM floor prevents the first term from certifying the target, but the BO term still gives

$$T_{\text{LB}}(\delta) = \tilde{\mathcal{O}}\left(\frac{1}{\epsilon}\left(\frac{C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}})}{\delta}\right)^{d_*}\right).$$

Define the acceleration ratio as

$$\rho(\delta) := \frac{T_{\text{Base}}(\delta)}{T_{\text{LB}}(\delta)}.$$

Ignoring logarithmic and lower-order root-allocation factors, we obtain

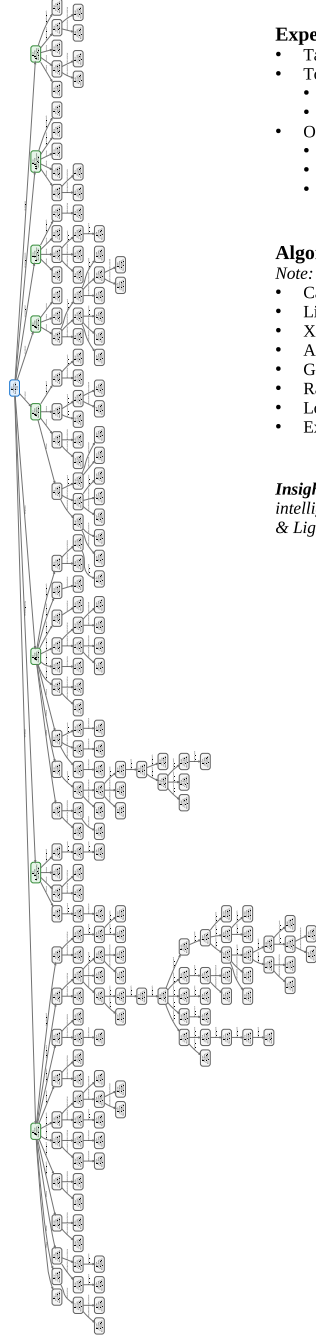
$$\rho(\delta) = \begin{cases} \max\left\{\left(\frac{C_{\text{EI}}}{\delta}\right)^{d_*} \left(\frac{\delta - \Delta_{\text{inf}}}{C_{\text{LLM}}}\right)^{1/\alpha}, \epsilon \left(\frac{C_{\text{EI}}}{C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}})}\right)^{d_*}\right\}, & \delta > \Delta_{\text{inf}}, \\ \epsilon \left(\frac{C_{\text{EI}}}{C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}})}\right)^{d_*}, & \delta \leq \Delta_{\text{inf}}. \end{cases} \quad (11)$$

Hence, in the BO-dominated regime, the finite-time acceleration comes from the high-quality LLM evaluations that warm-start the GP posterior, reducing the EI constant from C_{EI} to $C_{\text{EI}}(\mathcal{D}_q^{\text{LLM}})$ while preserving the EI rate in Eq. (8). In the LLM-dominated regime, additional gains arise from the LLM’s semantic prior and in-context learning capability, captured by the power-law term in Eq. (9).

Theoretical-Empirical Alignment. Empirically, LB-MCTS achieves an average $2.37\times$ acceleration over B-MCTS (which retains only the BO proposer) across 10 datasets in the BO–LLM collaboration ablation (Figure 6), confirming the practical benefit of LLM-assisted warm starts and semantic proposals.

A.4 A Case of MCTS Result

While Figure 1 in the main paper shows an abstract schematic of the MCTS search structure, Figure 7 presents a concrete example of an actual search tree generated by LB-MCTS on the “Adult” dataset. The visualization reveals how BO and the LLM proposer collaboratively expand the tree: The first leaf under each Algo node serves as the initial configuration for that path, established via either an LLM warm-start or BO random search. Along a typical optimization path, BO-driven local search and LLM-driven exploratory/exploitative proposals alternate in refining configurations.



Experiment Statistics

- Task: Classification (Dataset: adult)
- Total Nodes: N=309
 - 1 CASH Node + 8 Algo Nodes
 - 300 HPO Nodes
- Optimization Strategy:
 - Warmup: 24 trials
 - LLM(98 trials): Exploration(56) & Exploitation(42)
 - BO(178 trials): Random(46) & Local Search(132)

Algorithm Counts

Note: The visualization selectively displays dense trajectories for clarity.

- CatBoost: 129
- LightGBM: 81
- XGBoost: 27
- AdaBoost: 22
- Gradient Boosting: 11
- Random Forest: 10
- Logistic Regression: 10
- Extra Trees: 10

Insight: The tree structure reveals a dynamic search pattern. The method intelligently distributes 70% of the budget between the two top performers (CatBoost & LightGBM) while automatically assigning fewer trials to other candidates.

Figure 7: MCTS tree generated by LB-MCTS on the Adult dataset.

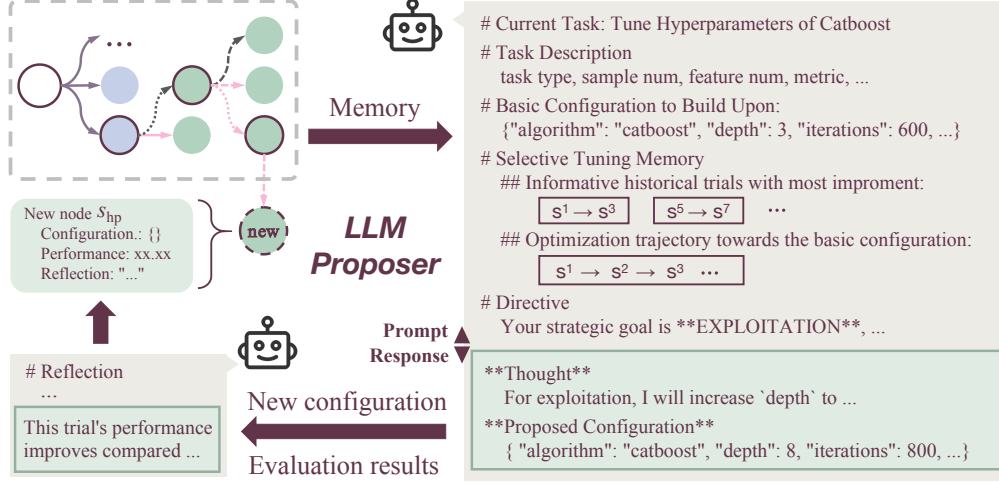


Figure 8: MCTS search of the LLM proposer.

A.5 LLM Proposer Search Illustration

The LLM proposer first selects a base node within the chosen algorithm subtree using PUCT, then expands it with an LLM-generated configuration, evaluates the new configuration, produces a reflection, and backpropagates the result to update the tree statistics.

B Prompt Design and Case Studies

B.1 Prompt Design For Hyperparameter Tuning Phase

The hyperparameter tuning process follows a dual-prompt mechanism: a static **System Prompt** defining the agent’s persona, and a dynamic **User Prompt** supplying the optimization context. The User Prompt is constructed using a template comprising four key components: (i) task description, (ii) dataset context, (iii) selective tuning memory, and (iv) output requirements. Crucially, the strategic directive within the task description is instantiated at runtime based on the MCTS state.

System Prompt:

You are an expert AutoML strategist. Your current role is **Hyperparameter Tuner** in a Combined Algorithm Selection and Hyperparameter optimization (CASH) task. An algorithm has been selected. Your goal is to adjust a basic hyperparameter configuration on a monte carlo tree search node to find the best hyperparameter configuration for it.

Template: User Prompt For Hyperparameter Tuning Phase

Task Description:

Your ultimate goal is to solve the Combined Algorithm Selection and Hyperparameter optimization (CASH) problem: finding the best possible algorithm and its optimal hyperparameters for a classification task on the provided dataset.

Algorithm ‘[ALGORITHM]’ has been selected. Your immediate objective is to propose a hyperparameter configuration for it.

The proposed configuration must strictly adhere to the defined search space:

- parameter_1: [TYPE], [RANGE/CHOICES]
- parameter_2: [TYPE], [RANGE/CHOICES]
- ...

The Basic Configuration to Build Upon:

This configuration is derived from previous trials and represents a promising starting point:

```
{ "parameter_1": [VALUE], "parameter_2": [VALUE], ... }
```

Your current strategic goal is [MODE]. [INSTRUCTION_TEXT]

(Note: This directive is dynamically instantiated based on the MCTS state. See "Strategy Variants" below.)

Dataset Context:

The target task is a [TASK_TYPE] problem evaluated on the provided dataset. The primary optimization metric is [METRIC]. The dataset consists of [N_SAMPLES] samples and [N_FEATURES] features...

Selective Tuning Memory:

This component integrates historical trajectories and high-performing trials to inform the current optimization step.

Optimization Trajectory Context:

[Sequence: Root → ... → Ancestral Node → Current Basic Node]

Informative Historical Trials:

[List of Top-k High-Performing & Similar Historical Trials]

Required Output Format:

You must provide your reasoning followed by the final configuration in the following structure.

Thought: ...

Action: { "parameter_1": [VALUE], ... }

Optimization Directives : Strategy Variants

The placeholders [MODE] and [INSTRUCTION_TEXT] in the template above are dynamically instantiated based on the MCTS search state.

Mode: WARMUP

Instruction: Propose a promising configuration **from scratch**. Focus on identifying configurations that are **distinct from previous attempts** while still demonstrating strong performance. Analyze historical data to **balance novelty and effectiveness**.

Mode: EXPLORATION

Instruction: Modify the basic configuration to **explore new areas** of the hyperparameter space. Focus on making changes that **introduce diversity** and help uncover potentially high-performing configurations. Make **bold adjustments** to discover new possibilities.

Mode: EXPLOITATION

Instruction: **Refine and optimize** the basic configuration to achieve peak performance. Focus on making **minor, incremental adjustments** to the basic configuration, guiding it towards a more optimal setup using insights from successful trials.

B.2 Prompt Design For Reflection Phase

Similar to the tuning phase, the reflection phase is governed by a dedicated System Prompt that establishes the ‘Performance Analyst’ persona. The User Prompt is structured to feed execution results back to the LLM, enabling it to perform quantitative attribution analysis and synthesize reusable insights.

System Prompt

You are an expert AutoML strategist. Your current role is **Performance Analyst** in a Combined Algorithm Selection and Hyperparameter optimization (CASH) task. Your goal is to analyze the results of a hyperparameter tuning trial. You must provide a quantitative, structured, and globally useful reflection that explains the relationship between parameter changes and performance outcomes to guide future trials.

Template: User Prompt For Reflection Phase

Task Description:

Your ultimate goal is to solve the Combined Algorithm Selection and Hyperparameter optimization (CASH) problem: finding the best possible algorithm and its optimal hyperparameters for a classification task on the provided dataset.

A hyperparameter configuration for algorithm ‘[ALGORITHM]’ has just been evaluated. Your immediate objective is to **Reflect on Trial Performance...**

The Trial to Analyze:

This section details the configuration changes and the resulting performance shift relative to the basic configuration:

Basic Configuration { "parameter": [VALUE], ... }

Basic Performance [BASIC_PERF]

Original Rationale [AGENT_THOUGHT_FROM_TUNING_PHASE]

Resulting Configuration { "parameter": [VALUE], ... }

Resulting Performance [NEW_PERF] ([CHANGE_%] relative to basic)

Global Ranking [RANK] out of [TOTAL_TRIALS] trials

Analytical Reflection Requirements:

Your analysis must be “quantitative, structured, and globally useful”:

- In Reflection:
 1. Triage & Compare: Position this trial among historical data...
 2. Attribute & Synthesize: Explain which parameter deviation led to the outcome...
- In Reflection Summary: Distill into a high-confidence summary of global lessons. (Only this summary will be stored in memory).

Dataset Context:

The target task is a [TASK_TYPE] problem evaluated on the provided dataset. The primary optimization metric is [METRIC]. The dataset consists of [N_SAMPLES] samples and [N_FEATURES] features...

Selective Tuning Memory:

[Identical to the Tuning Phase Input: Trajectory Context + Historical Trials]

Required Output Format:**Reflection:**

(Your structured analysis addressing the two points above...)

Reflection Summary:

(A compact, single paragraph summary of global, reusable and high-confidence conclusions...)

B.3 Reflection Design for BO and LLM WARMUP

While the LLM proposer follows each recommendation with a dedicated reflection stage, the BO proposer lacks an inherent mechanism for semantic reasoning. Similarly, LLM WARMUP trials are excluded from LLM-based reflection because the sparse early-stage history provide insufficient observations and lack a reference “basic configuration” for meaningful comparison. To bridge this gap and maintain a consistent search history, we employ fixed programmatic rules to automatically summarize these trials.

Reflection of BO LOCAL SEARCH. The BO LOCAL SEARCH generates candidates by sampling from the neighborhood of a basic configuration via small perturbations. Since this process is inherently incremental, its reflection is designed to explicitly capture these fine-grained modifications by concatenating two programmatic components:

1. **Parameter Change Description:** Lists all hyperparameters modified from the parent configuration, formatted as “Changed parameters: {param_name}: {old} -> {new}”.
2. **Performance Change Description:** Quantifies the performance shift using the computed delta, formatted as “Performance {improved|declined} from {parent_perf} to {current_perf}”.

Example Case: Consider a LOCAL SEARCH trial on the cmc classification dataset. The optimization transition is defined by the following parent and child nodes:

- Parent Node:
 - Performance: 0.5846677
 - Configuration: {"algorithm": "SAMME", "learning_rate": 4.049957e-02, "max_depth": 5, "n_estimators": 217}
- Child Node:
 - Performance: 0.59120365
 - Configuration: {"algorithm": "SAMME", "learning_rate": 4.000687e-02, "max_depth": 5, "n_estimators": 231}

Based on this transition, the system automatically generates the following textual reflection, which serves as part of the context for future LLM iterations:

System Generated Reflection

```
"Bayesian Local search performed; Changed parameters: learning_rate:
4.049957e-02 -> 4.000687e-02, n_estimators: 217 -> 231. Performance
improved from 0.5846677 to 0.59120365."
```

Special Cases: BO RANDOM SEARCH and LLM WARMUP:

For WARMUP trials and BO RANDOM SEARCH trials (where the parent node is an HPORootNode), the reflection string is set to “Warmup configuration.” Moreover, instead of a performance change description, the system appends “Initial performance: {current_perf}” to indicate this is the first trial of an optimization trajectory.

Notably, if WARMUP or BO RANDOM SEARCH trials achieve exceptionally high performance and are selected by the Pareto frontier, they may appear in the Informative Historical Trials section of the prompt. In such cases, only the Child configuration and Reflection are displayed (no Parent), since the parent is an HPORootNode.

B.4 Prompt Example: Execution Trace

This section presents a concrete execution trace from the cmc classification dataset experiment. It displays the raw input prompts and the corresponding model outputs during a **Hyperparameter Tuning** step and its subsequent **Reflection** step.

User Prompt For Hyperparameter Tuning

Task Description:

Your ultimate goal is to solve the Combined Algorithm Selection and Hyperparameter optimization (CASH) problem: finding the best possible algorithm and its optimal hyperparameters for a classification task on the provided dataset.

Algorithm “adaboost” has been selected. Your immediate goal is to propose a hyperparameter configuration for it.

The proposed configuration must strictly adhere to the defined search space:

- `algotirhm`: Categorical, {SAMME.R, SAMME}
- `learning_rate`: Float, Range=[1e-02, 2e+00] (Log-Scale)
- `max_depth`: Integer, Range=[2, 8]
- `n_estimators`: Integer, Range=[50,500]

The Basic Configuration to Build Upon:

This configuration is derived from previous trials and represents a promising starting point:

```
{"algorithm": "SAMME", "learning_rate": 4.5e-02, "max_depth": 5,
"n_estimators": 350}
```

Your current strategic goal is **Exploitation**. Refine and optimize the basic configuration to achieve peak performance. Focus on making minor, incremental adjustments to the basic configuration, guiding it towards a more optimal setup using insights from successful trials.

Dataset Context:

The target task is a **3-class classification** problem evaluated on the provided dataset. The primary optimization metric is **balanced accuracy**, where a higher value indicates superior performance. The dataset consists of **1,178 samples** and **9 total features**, partitioned into 9 numeric and 0 categorical features. The target classes are distributed as **42.7%** for Class 1, **34.7%** for Class 3, **22.6%** for Class 2.

Selective Tuning Memory:

This component integrates historical trajectories and high-performing trials to inform the current optimization step.

Optimization Trajectory Context:

This sequence preserves the ancestral path from the algorithm root, providing trajectory awareness:

...

Ancestral Node { "performance": 0.58031504, "configuration":
{"algorithm": "SAMME", "learning_rate": 3.9e-02, "max_depth":
5, "n_estimators": 265}, "reflection": "..."} }

Current Basic Node { "performance": 0.57471745, "configuration":
{"algorithm": "SAMME", "learning_rate": 4.5e-02, "max_depth":
5, "n_estimators": 350}, "reflection": "Performance
degradation (-0.97%) indicates overfitting caused by
simultaneously increasing learning_rate (0.039->0.045) and
n_estimators (265->350). Future tuning must constrain the
learning rate when scaling up model complexity."} }

Informative Historical Trials:

The following trials represent top-performing historical attempts that are most similar to the current configuration:

Historical Trial A(Proposed by BO Local Search):

- **Parent:** { "performance": 0.5846677, "configuration": {"algorithm": "SAMME", "learning_rate": 4.049957e-02, "max_depth": 5, "n_estimators": 217} }
- **Child:** { "performance": 0.59120365, "configuration": {"algorithm": "SAMME", "learning_rate": 4.000687e-02, "max_depth": 5, "n_estimators": 231} }
- **Reflection:** "Decreasing learning_rate (0.04049957->0.04000687) and increasing n_estimators (217->231) improved performance by 0.0065 (+1.12%)."

Historical Trial B(Proposed by LLM):

- **Parent:** { "performance": 0.58264467, "configuration": {"algorithm": "SAMME", "learning_rate": 4.1953e-02, "max_depth": 5, "n_estimators": 233} }
- **Child:** { "performance": 0.58918061, "configuration": {"algorithm": "SAMME", "learning_rate": 3.8e-02, "max_depth": 5, "n_estimators": 280} }
- **Reflection:** "Reducing the learning rate to 0.038 successfully mitigated oscillation, creating the stability required to profitably scale n_estimators from 233 to 280. This confirms that higher model capacity demands a stricter learning rate constraint."

...

Required Output Format:

You must provide your reasoning followed by the final configuration in the following structure:

Thought:

(Provide a detailed rationale. Analyze the **Optimization Trajectory Context** and the **Informative Historical Attempts** and connect your insights with the **Dataset Summary** and the current **Directive** to justify your parameter adjustments.)

Action:

Output the configuration as a single, valid JSON block.

```
{ "parameter_1": value_1, "parameter_2": value_2, ... }
```


ASSISTANT Response (Tuning)

Thought:

The current directive is **EXPLOITATION**, which requires refining the basic configuration based on historical insights.

Analyzing the *Optimization Trajectory Context*, the current basic node (LR 0.045, Est 350) suffered a performance drop due to overfitting. The reflection explicitly warns that the learning rate must be constrained when scaling up estimators.

Looking at *Informative Historical Trials*, specifically "Historical Trial B", we see a clear success pattern where a lower learning rate of 0.038 enabled the effective scaling of estimators to 280. This aligns perfectly with the trajectory's lesson.

Therefore, to exploit this finding and optimize performance: 1. I will revert the `learning_rate` to the proven stable value of **0.038**. 2. I will set `n_estimators` to **300**. This is a precise adjustment that sits between the under-fitted 265 and the over-fitted 350, slightly pushing the boundary of the successful Trial B (280) to maximize model capacity within the safe learning rate regime. 3. I will maintain the algorithm as "SAMME" and `max_depth` at 5, as these are established optima.

Action:

```
{
  "algorithm": "SAMME",
  "learning_rate": 0.038,
  "max_depth": 5,
  "n_estimators": 300
}
```

User Prompt For Reflection

Task Description:

Your ultimate goal is to solve the Combined Algorithm Selection and Hyperparameter optimization(CASH) problem: finding the best possible algorithm and its optimal hyperparameters for a classification task on the provided dataset.

A hyperparameter configuration for algorithm “**adaboost**” has just been evaluated. Your immediate objective is to Reflect on Trial Performance. You must analyze the results of the recent trial to synthesize quantitative, structured, and reusable lessons for future tuning.

The Trial to Analyze:

This section details the configuration changes and the resulting performance shift relative to the basic configuration:

Basic Configuration {"algorithm": "SAMME", "learning_rate": 4.5e-02, "max_depth": 5, "n_estimators": 350}

Basic Performance 0.57471745

Original Rationale “The current directive is EXPLOITATION... [omitted detailed analysis] ... ”

Resulting Configuration {"algorithm": "SAMME", "learning_rate": 3.8e-02, "max_depth": 5, "n_estimators": 300}

Resulting Performance 0.59244859 (improved by 3.09% relative to the basic configuration)

Global Ranking 1 out of 219 trials of ‘adaboost’

Analytical Reflection Requirements:

Your analysis must be “quantitative, structured, and globally useful”:

- **In Reflection:**

1. **Triage & Compare:** Position this trial among historical data using numerical comparisons (absolute differences and percentages).

2. **Attribute & Synthesize:** Explain which parameter deviation led to the outcome.
 - Highlight **clearly important** parameters, **value ranges** that look promising/risky, and any **interactions** inferred from numbers.
 - Make reasoning **reusable** to guide future tuning (not just describing this single trial).
 - Only include conclusions you are **confident about** based on quantitative evidence.
- **In Reflection Summary:** Distill into a high-confidence summary of global lessons.
 - Include only insights supported by multiple numerical observations.
 - Do **not** invent rules for noisy data; do **not** suggest explicit "next steps" or new configurations.

Dataset Context:

The target task is a **3-class classification** problem evaluated on the provided dataset. The primary optimization metric is **balanced accuracy**, where a higher value indicates superior performance. The dataset consists of **1,178 samples** and **9 total features**, partitioned into 9 numeric and 0 categorical features. The target classes are distributed as **42.7%** for Class '1', **34.7%** for Class '3', **22.6%** for Class '2'.

Selective Tuning Memory:

... (For brevity, the Selective Tuning Memory content is omitted as it remains identical to the Tuning Phase input) ...

Required Output Format:

Reflection:

(Your structured analysis addressing the two points above, focused on quantitative comparison and attribution of the outcome.)

Reflection Summary:

(A compact, single paragraph summary of global, reusable and high-confidence conclusions derived from this analysis.)

ASSISTANT Response (Reflection)

Reflection:

1. **Triage & Compare:** The trial achieved a score of **0.5924**, improving by **3.09%** over the basic configuration (0.5747). Ranking **1st out of 219**, this confirms the configuration is in the optimal region.
2. **Attribute & Synthesize:** The primary driver of success was reducing the `learning_rate` from 0.045 to **0.038**. As hypothesized in the rationale, the previous failure at LR 0.045 was due to overfitting when combined with high estimators. By constraining the LR to 0.038, the model successfully accommodated the `n_estimators` (300), validating the interaction pattern observed in Historical Trial B. The stability of `max_depth=5` further supported this gain.

Reflection Summary:

For 'adaboost' on the provided dataset, optimal performance is achieved by **constraining the learning rate to ≈ 0.038** when scaling `n_estimators` (280-300). The interaction is critical: higher capacities require stricter learning rate constraints to prevent overfitting. **max_depth** should be maintained at 5.

C Results and Analysis

C.1 Search Space

We evaluate LB-MCTS across both classification and regression tasks using a search space adapted from the VolcanoML AutoML system [60]. Specifically, we refined the candidate pool by pruning less common models, ultimately retaining eight representative algorithms for each task type to ensure both diversity and efficiency (see Table 1). These selections span major model families, including tree ensembles (e.g., Random Forest, Extra Trees), boosting frameworks (e.g., XGBoost, LightGBM), and linear models (e.g., Logistic Regression, Ridge).

Table 1: Search space for ML algorithms. We distinguish categorical (cat) hyperparameters from numerical (cont) ones.

Type of Classifier	# λ	cat	cont	Type of Regressor	# λ	cat	cont
AdaBoost	4	1	3	AdaBoost	4	1	3
Random Forest	5	2	3	Random Forest	5	2	3
Extra Trees	5	2	3	Extra Trees	5	2	3
Gradient Boosting	7	1	6	Gradient Boosting	7	1	6
Logistic Regression	4	2	2	Ridge	4	1	3
LightGBM	7	-	7	LightGBM	7	-	7
Catboost	4	-	4	Catboost	5	1	4
Xgboost	9	-	9	Xgboost	9	-	9
Total (8 algos)	45	8	37	Total (8 algos)	46	8	38

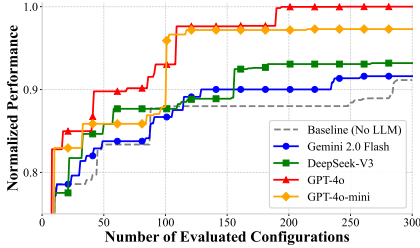


Figure 9: Average performance on 2 datasets.

Table 2: Ablation study of different LLM backbones on the ada (classification) and quake (regression) datasets. Cost is reported in USD. The best performance for each dataset is bolded, and the second-best results are underlined.

LLM Backbone	ada		quake	
	Bal Acc. $\uparrow$	Cost (\$)	MSE $\downarrow$	Cost (\$)
No LLM (Pure BO)	0.8086	0.00	0.03475	0.00
Gemini 2.0 Flash	0.8078	0.11	0.03462	0.06
GPT-4o-mini	<u>0.8126</u>	0.14	<u>0.03458</u>	0.11
DeepSeek V3	0.8102	0.23	0.03472	0.14
GPT-4o	0.8142	3.81	0.03449	2.85

For each algorithm, we construct a comprehensive hyperparameter search space involving a mix of continuous and categorical parameters. As shown in the table, the classification setting involves a total of 45 hyperparameters (37 continuous, 8 categorical) across the 8 algorithms, while the regression setting comprises 46 hyperparameters (38 continuous, 8 categorical). This formulation constitutes a high-dimensional, hierarchical CASH problem, where the optimizer must first select an algorithm and then navigate its specific conditional hyperparameter subspace.

C.2 Ablation on Different LLM Backbones

This appendix further studies how the choice of LLM backbone affects LB-MCTS. We compare four LLMs (GPT-4o [62], GPT-4o-mini [32], DeepSeek-V3 [56], and Gemini-2.0-Flash [58]) and a No LLM variant that removes the LLM proposer and uses only the BO proposer. Due to the substantial API costs associated with premium models (for instance, GPT-4o can incur expenses of several USD per individual task), experiments are run on one classification task (ada) and one regression task (quake) to cover both metric types while maintaining a manageable budget.

Figure 9 reports the normalized best-so-far validation performance versus the number of evaluations. The most capable (and most expensive) GPT-4o consistently achieves the highest curve, with GPT-4o-mini close behind. Importantly, *all* LLM-based variants outperform the pure-BO baseline, confirming that our integration framework can effectively exploit LLM guidance to complement Bayesian Optimization (BO) regardless of the specific backbone.

Table 2 summarizes the final performance and cost. GPT-4o represents the performance upper bound, achieving the best results on both datasets (0.8142 Balanced Accuracy and 0.03449 MSE). However, it is also the most expensive, costing significantly more than other backbones. Conversely, Gemini 2.0 Flash is the most affordable option but yields the lowest performance among the LLM variants. GPT-4o-mini emerges as a highly cost-effective choice. It secures the second-best performance ranking while maintaining a low cost (e.g., 0.14 for the “ada” task compared to GPT-4o’s 3.81). Its performance-to-price ratio makes it a practical recommendation for budget-constrained large-scale CASH tasks.

To sum up, the results indicate that the “ceiling” of our proposed method is directly correlated with the reasoning capabilities of the underlying LLM. As the field of LLM research continues to evolve—leading to models with enhanced reasoning capabilities and reduced inference costs—the

efficacy and accessibility of our framework are expected to scale proportionally, making it increasingly viable for a wider range of applications.

C.3 Implementation Details of Baselines

We adhered to the open-source versions or the methodologies outlined in the original papers for all other baseline implementations. For consistency, we implement the BO components of all baselines with Openbox 0.8.1 [17], an open-source toolkit designed for black-box optimization. We employ the same LLM backbone, GPT-4o-mini [32], for all LLM-based optimizers to eliminate differences due to model strength. Since prior LLM-based methods typically use only a few initial configurations (e.g., ~ 5 in LLAMBO and BORA) on much simpler tasks, we standardize and strengthen warm-starting for our harder CASH setting. Accordingly, to ensure fairness, all LLM-based baselines use the same number of warm-start configurations as our method (3 LLM-generated initial configurations per algorithm). Meanwhile, pure BO-based baselines follow their standard initialization protocols, as they are natively designed for the CASH setting. All the experiments are conducted on a machine with 24 ‘AMD EPYC 7702P’ CPU cores and 80G memory.

- **SMAC [15]**. We implement SMAC as a standard BO-based CASH optimizer using a random-forest surrogate. The optimizer treats CASH as a structured configuration space with conditional hyperparameters, and iteratively proposes one configuration per evaluation.
- **OptDivBO [33]**. OptDivBO enhances standard BO by encouraging diversity in the proposed models through ensemble selection. Following the original paper, we fix the diversity trade-off parameter τ at 0.2 and set the ensemble size for post-hoc selection to 25.
- **Rising Bandit(RB) [23]**. RB addresses the CASH problem by treating algorithm selection as a multi-armed bandit problem. If the upper bound of the reward of an arm k (algorithm A^k) is less than or equal to the lower bound of another arm in the candidate set, then the arm k will be eliminated from the candidate set. The parameter C for computing the smooth growth rate is set to 7.
- **MOSAIC [34]**. MOSAIC uses a tree policy for algorithm selection and BO for HPO. Algorithm selection is guided by a UCB-style criterion with exploration constant $C_{\text{ucb}} = 1.3$, and tree expansion is controlled by progressive widening with coefficient $PW = 0.6$. For the BO component, MOSAIC fits a *single global* random-forest surrogate over the entire CASH space using all observed trials (rather than per-algorithm surrogates), and proposes configurations under the selected algorithm accordingly.
- **OPRO [48]**. OPRO treats LLM as a standalone optimizer. In each iteration, we prompt the LLM with the task description and a truncated history containing only the top-20 best-performing trials found so far. The LLM is instructed to identify patterns from these high-quality examples and propose a single new configuration for evaluation.
- **LLAMBO [25]**. LLAMBO replaces traditional BO components with LLM. Following the official implementation, we include the entire optimization history in the prompt (subject to context limits). In each round, the LLM is tasked to generate candidates that improve upon the current best score. The method generates 20 candidate configurations per iteration. It then randomly shuffles the order of historical examples 10 times to aggregate the predictions. Consequently, each optimization round necessitates a total of 30 API calls.
- **BOPRO [1]**. BOPRO uses BO to optimize the prompt instructions themselves. For the in-context learning component, the prompt includes a set of 8 historical examples. In each iteration, the BO-selected instruction template guides the LLM to reason over these examples and generate a new hyperparameter configuration.
- **BORA [4]**. BORA dynamically switches between an LLM and a GP-BO proposer based on posterior variance. When the LLM is active, we follow the original paper’s setting by including the full optimization history in the prompt to maximize context awareness.

C.4 Impact of Task Characteristics

We further analyze how the performance of LB-MCTS correlates with task characteristics by examining the final ranks after 300 iterations across all 104 datasets. Figure 10 visualizes the relationship between the validation rank and three key factors: task type, sample size, and feature dimensionality.

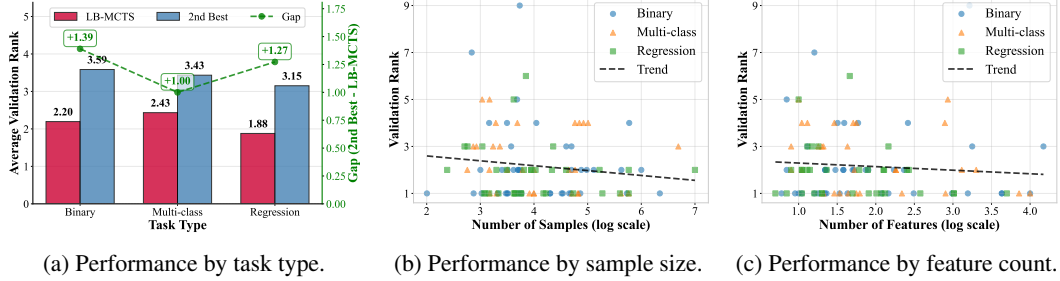


Figure 10: Performance vs. task characteristics across 104 datasets.

Table 3: Search space for DL models. The same space is used for classification and regression tasks.

Type of Model	# λ	cat	cont
MLP	5	3	2
ResNet-Tabular	7	4	3
FT-Transformer	9	6	3
TabNet	10	3	7
Total (4 algos)	31	16	15

Impact of Task Type. Figure 10a compares the average rank of LB-MCTS against the second-best baseline across Binary Classification, Multi-class Classification, and Regression tasks. LB-MCTS consistently outperforms the runner-up with a significant rank advantage across Binary (+1.39), Multi-class (+1.00), and Regression (+1.27) tasks. Notably, it achieves the best performance on Regression tasks (Rank 1.88). This may be attributed to the continuous nature of the MSE metric used in regression, which provides smoother gradient-like feedback for the LLM’s reflection mechanism compared to the discrete nature of Balanced Accuracy in classification.

Impact of Problem Complexity. Figure 10b and Figure 10c plot the validation rank against the $\log_{10}$ -scale sample size and feature count, respectively, with fitted trend lines. We observe a clear downward trend in rank (indicating better performance) as task complexity increases. To quantify this, we partition the datasets into three equal-sized groups (Small/Medium/Large) based on sample size or feature count respectively.

- **Sample Size:** The average rank progressively improves from 2.31 on small datasets ($n < 10^{3.54}$) to 2.24 on medium datasets ($10^{3.54} \leq n < 10^{4.53}$), and reaches **1.94** on large datasets ($n \geq 10^{4.53}$).
- **Feature Count:** Similarly, the rank improves from 2.37 on low-dimensional tasks ($p < 10^{1.33}$) to 2.15 on medium-dimensional tasks ($10^{1.33} \leq p < 10^{2.05}$), and achieves **1.97** on high-dimensional tasks ($p \geq 10^{2.05}$).

Overall, across all 104 datasets, LB-MCTS demonstrates robust dominance, achieving Rank 1 on 43 tasks and Rank ≤ 3 on 87 tasks. These results suggest that LB-MCTS is particularly adept at handling complex, large-scale optimization problems where traditional methods often struggle to scale.

C.5 Supplementary Experiments on Deep Learning and Feature Engineering

Furthermore, we conducted supplementary experiments on a more diverse search space including both deep learning (DL) models and feature engineering (FE). Specifically, we considered four DL models, MLP, ResNet-Tabular and FT-Transformer [59], and TabNet [52], forming a 31-dimensional model hyperparameter space (Table 3), together with a 10-dimensional FE space. In each model subtree, LB-MCTS jointly optimizes the model hyperparameters together with the 10-dimensional FE space. This search space is shared by classification and regression tasks, and jointly optimizing FE with each model is natural since effective feature engineering is often model-dependent.

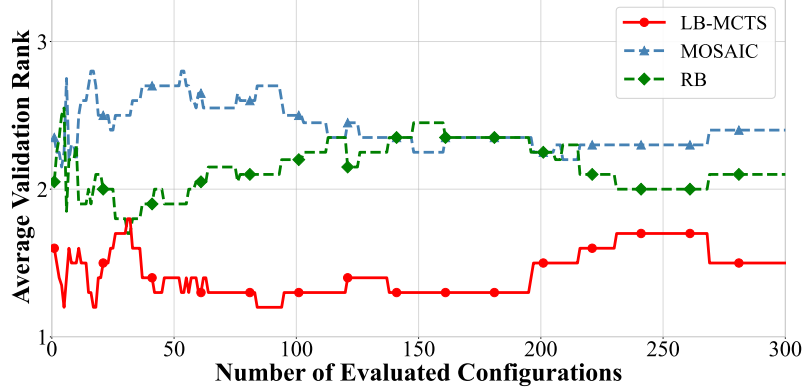


Figure 11: Average validation rank over 10 datasets on the deep learning pipeline scenario.

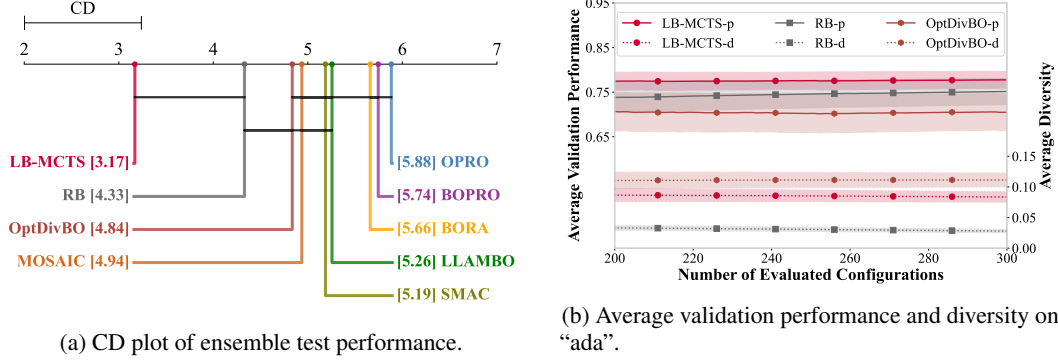


Figure 12: Post-hoc ensemble performance of 9 methods across 104 datasets.

Results across 10 representative datasets in Figure 11 show that LB-MCTS achieves the best average rank of 1.5, outperforming the top two baselines from our main experiments. This confirms that our framework consistently excels at searching within structured spaces, regardless of the underlying model type.

C.6 Post-hoc Ensemble Performance

In practical applications, an ensemble of promising configurations is sometimes preferred over a single best model, as ensembling can potentially enhance overall performance [57, 64]. To evaluate this auxiliary aspect, we perform post-hoc ensembling using the pool of 300 models identified during the optimization process (without retraining). Among different ensemble strategies (e.g., Bagging, Boosting, Stacking), we adopt the ensemble selection [55], which works empirically well with AutoML as shown in previous studies [60] and [37, 33]. In short, ensemble selection starts from an empty ensemble and iteratively adds models from the pool with replacement to maximize the ensemble validation performance (with uniform weights).

Figure 12a presents the *test performance after ensemble selection* on 104 datasets. Two points emerge: (i) Notably, OptDivBO rises from the lowest-ranking BO baseline (in single-model settings) to the second-best position. This validates that effective ensembling requires not just strong individual learners but also diversity among them [53, 66], which aligns precisely with OptDivBO’s motivation of considering model diversity. (ii) LB-MCTS maintains its leading position, achieving **an average rank of 3.17**—a significant margin over the second-best performing method (4.33). This is attributed to its explicit balance of inter- and intra-algorithm exploration and exploitation; LB-MCTS not only identifies high-performing individual models but also ensures broad coverage, effectively achieving the trade-off between individual model performance and diversity among models.

To support this interpretation, Fig. 12b reports, on the “ada” dataset, how (i) the *average validation performance* (solid lines) of all evaluated models and (ii) the *average inter-model diversity* (dashed lines) evolve over the course of optimization for the three methods with the best average ensemble

performance. Following [65] and [37], we quantify the diversity between two models (λ_i, λ_j) as the average Euclidean distance between their predicted class-probability vectors on the validation set:

$$\text{Div}(x_i, x_j) = \frac{\sqrt{2}}{2} \cdot \frac{1}{|D_{\text{val}}|} \sum_{s \in D_{\text{val}}} \|T_{\lambda_i}(s) - T_{\lambda_j}(s)\|_2,$$

where $T_x(s)$ denotes the class-probability prediction of model x on validation example s . We can observe from the figure that OptDivBO yields the highest average diversity among the three models, but its average validation performance is substantially worse than the other two methods, resulting in an extreme “diverse-but-weak” pool. RB shows the opposite behavior: it achieves the second-best average validation performance but produces the least diverse model set. In contrast, LB-MCTS attains the best average validation performance while maintaining high diversity (second only to OptDivBO), providing a more balanced and thus more effective pool for post-hoc ensemble selection, which aligns with its strongest ensemble results. We note that this balance arises as a by-product of LB-MCTS’s exploration–exploitation design; explicitly optimizing for ensemble performance is left for future work.

C.7 LLM Cost Comparison

To comprehensively evaluate the practicality of LB-MCTS, we analyze the cost from two perspectives: the monetary cost of LLM API usage and the temporal overhead of the optimizer. Figure 13 summarizes the results across 104 datasets.

Monetary Cost vs. Performance. We estimate API cost using model pricing at the time of experiments, accounting for input and output tokens. Figure 13a illustrates the final average validation rank (across 104 datasets after 300 rounds) versus the average LLM API cost per task (in USD). The results highlight several key insights: (i) LB-MCTS occupies the ideal bottom-left quadrant of the plot, achieving a state-of-the-art **average rank of 1.25** with an **economical cost of approximately \$0.127 per task**. Compared to other baselines, LB-MCTS provides a significant performance leap without a proportional increase in expenditure. (ii) While LLAMBO attains the second-best rank (2.73), it is by far the most expensive method, costing \$0.860 per task—nearly $6.8\times$ that of LB-MCTS. This high cost stems from its high-volume API calls per iteration, which do not translate into superior rankings compared to our tree-search approach. (iii) Methods like BOPRO, OPRO, and BORA cluster in the low-cost region (\$0.11–\$0.18) but suffer from poor optimization efficacy, with average ranks exceeding 3.20. Although these methods are budget-friendly, their inability to effectively navigate complex search spaces limits their utility.

Optimizer Time Overhead. Figure 13b reports the ratio of time spent on optimization (proposal generation) to the total runtime (optimization + model evaluation) over 300 iterations. Ideally, the optimizer should be lightweight, directing the majority of the time budget toward model training and evaluation. The results highlight several key insights: (i) LB-MCTS maintains a low overhead with a median ratio of 0.20, meaning $\approx 80\%$ of the time is productively spent on evaluation. This is significantly more efficient than other LLM-based approaches like OPRO (0.2356) and BORA (0.5841). The most extreme examples are LLAMBO (0.80), which suffers from excessive API calls, and BOPRO (0.81), which is slowed down by embedding model computations for all configurations. (ii) Scalability to Complex Tasks: Intuitively, as the dataset scale increases, model evaluation time grows, naturally reducing the optimizer’s relative overhead. Indeed, for large-scale tasks, LB-MCTS’s overhead ratio drops as low as 0.008. Crucially, this aligns perfectly with our finding in Appendix C.4 that LB-MCTS’s performance advantage is most pronounced on large, complex datasets, making it highly suitable for real-world, high-stakes optimization scenarios. (iii) From a wall-clock perspective, in our GPT-4o-mini benchmark runs, the average per-iteration time costs are 13.78s for LLM inference and 2.56s for BO computation, while the black-box evaluation (model training and validation) averages 63.02s. The LLM overhead is therefore worthwhile because better proposals reduce the number of expensive evaluations needed: LB-MCTS achieves average end-to-end wall-clock speedups of $2.0\times$ over RB and $2.2\times$ over MOSAIC, the second- and third-ranked baselines, respectively. (iv) Finally, we note that as LLM inference speeds continue to improve, the temporal overhead of LB-MCTS will further decrease, enhancing its applicability to time-sensitive tasks.

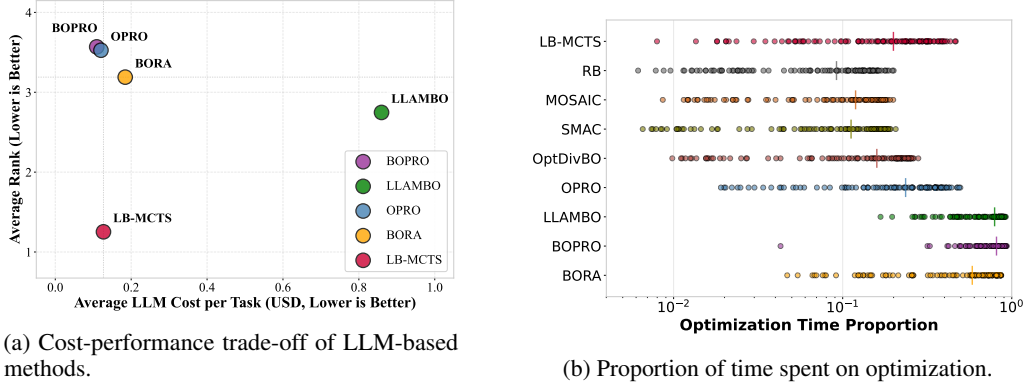


Figure 13: Comprehensive Cost and Performance Analysis.

Table 4: Overview of the 10 AMLB datasets selected for ablation studies.

Task Type	Dataset Name (OpenML ID)	Samples (n)	Features (p)	Classes (C)
Binary Classification	ada (190411)	4,147	49	2
	blood-transfusion (359955)	748	5	2
	sylvine (359972)	5,124	21	2
	qsar-biodeg (359956)	1,055	41	2
Multi-class Classification	cmc (359959)	1,473	9	3
	GesturePhaseSegmentation (359970)	9,873	32	5
	connect-4 (359977)	67,557	42	3
Regression	house_prices_nominal (359951)	1,460	79	-
	pol (359946)	15,000	48	-
	quake (359930)	2,178	3	-

C.8 Representative Dataset Selection for Ablation Study

Conducting hyperparameter optimization (HPO) with Large Language Models (LLMs) incurs substantial computational overhead and API costs. To balance experimental rigor with resource constraints, we selected a representative subset of **10 datasets** from the AutoML Benchmark (AMLB) suite for our ablation studies. As summarized in Table 4, these datasets were carefully chosen with three criteria: (i) Diverse task types: Including Binary Classification (4), Multi-class Classification (3), and Regression (3). This distribution is aligned with that of the original AMLB benchmark. (ii) Varying data scale: Sample sizes (n) ranging from 748 to 67,557. (iii) Varying feature dimensionality: Feature counts (p) ranging from 4 to 80. This selection ensures that the observed performance and search behaviors are robust across different data distributions and complexities. We have verified that performance trends of baselines on this subset are consistent with those on the full 104-dataset benchmark. As all the datasets are collected from OpenML [63], we provide the OpenML ID as the identification of the dataset.

C.9 Ablation Study: Exploration and Exploitation Strategy

Implementation details of LB-Flat. To isolate the contribution of the hierarchical tree structure and trajectory-aware reasoning, we implement LB-Flat, a variant that collapses the MCTS into a two-level structure (Root and Algo Nodes). It treats all evaluated configurations and performance within an algorithm’s subspace as an unstructured history. The BO proposer remains unaffected, as it still suggests candidates via random sampling and local refinement of historical observations. Similarly, the Proposer Selection mechanism is unchanged, continuing to calculate based on the BO surrogate model’s ability to fit the existing observations for each algorithm. In contrast, the LLM proposer is significantly modified: it no longer employs PUCT rules to select basic node configurations for refinement via exploration or exploitation directives, but prompts the LLM to directly recommend the next configuration. Due to the absence of optimization trajectories, Global and Local memories are discarded; instead, the top-20 historical observations are used as memory (following the OPRO approach). Finally, the reflection prompt is adjusted by removing all information related to basic configurations.

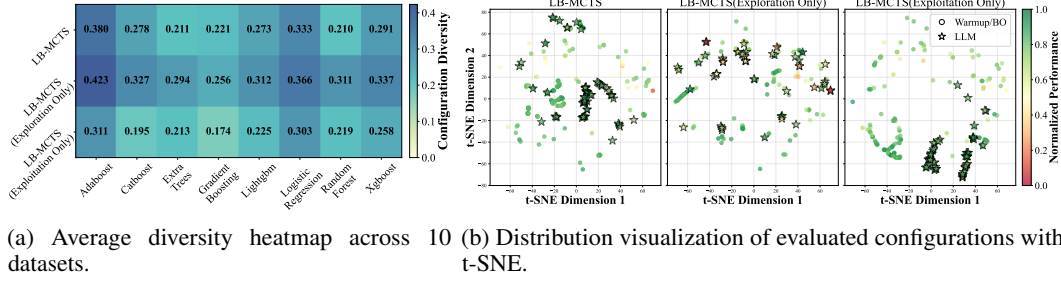


Figure 14: Visualizations for the Exploration-Exploitation Ablation Study. (a) compares the hyperparameter diversity, showing LB-MCTS achieves a balance between the extremes. (b) visualization of dominant algorithm “Adaboost” on “qsar-biodeg”

Tuning behaviors of EXPLORATION and EXPLOITATION. This appendix extends the ablation study on LB-MCTS’s EXPLORATION and EXPLOITATION strategies by investigating their impact on the distribution of proposed configurations. We first analyze the *diversity of proposed configurations*. Unlike our previous analysis of ensemble diversity, which considered the predictive differences between models on the validation set, here we quantify diversity directly at the level of the proposed hyperparameter configurations themselves. This allows for a more direct behavioral analysis of how different strategies shape the search space. Specifically, the diversity score is calculated independently for each algorithm. For a given algorithm A^i with a hyperparameter space Λ^i , the overall diversity score $D(A^i)$ is defined as the mean of the normalized diversity scores of all its hyperparameters: $D(A^i) = \frac{1}{|\Lambda^i|} \sum_{p \in \Lambda^i} S(p)$, where $S(p)$ denotes the normalized diversity score for a specific parameter p . The calculation of $S(p)$ depends on the parameter type (numeric or categorical).

- **Numeric Parameters.** For continuous or integer parameters (e.g., learning rate, max depth), we normalize sampled values to $x' \in [0, 1]$ using linear or log-domain min-max scaling as appropriate. The diversity score $S_{num}(p)$ is then defined as the standard deviation of these normalized samples: $S_{num}(p) = \text{std}(\{x'_i\})$.
- **Categorical Parameters.** We measure categorical diversity using normalized Shannon entropy: $S_{cat}(p) = \frac{-\sum_{k=1}^K q_k \ln(q_k)}{\ln(K)}$, where q_k is the empirical probability of category k among K possibilities. This ensures $S_{cat}(p) \in [0, 1]$, with 1 representing a perfectly uniform distribution (maximum diversity).

Figure 14a presents the heatmap of diversity scores by algorithm across all 10 datasets. Note that diversity is quantified per algorithm rather than across algorithms, preventing bias from different hyperparameter dimensionalities. We can observe that the EXPLORATION Only strategy (middle row) exhibits the highest diversity across almost all algorithms, indicating a broad but potentially inefficient search that mimics random sampling. The EXPLOITATION Only strategy (bottom row) shows the lowest diversity, suggesting rapid convergence to a narrow region of the hyperparameter space, carrying the risk of getting trapped in local optima. LB-MCTS (top row) maintains an intermediate diversity level. This confirms that our method effectively balances the trade-off, exploring sufficient configuration space before focusing resources on promising regions. To further illustrate how these diversity patterns arise, Figure Figure 14b visualizes the evaluated configurations for the AdaBoost algorithm on the “qsar-biodeg” dataset using t-SNE, with LLM-proposed points marked as stars and warmup/BO proposals as circles. Under the EXPLORATION Only strategy (middle), LLM proposals are widely scattered over the space, with many mediocre or poor configurations. Conversely, the EXPLOITATION Only case (right) shows LLM proposals tightly clustered in a single area; while the performance is acceptable, the search fails to expand beyond this local neighborhood, leaving the majority of the space unexplored. In contrast, LB-MCTS (left) generates a distribution that forms distinct, high-performing clusters (dark green stars) while retaining separation between them. This demonstrates that LB-MCTS successfully guides the search to exploit promising modes deeply while periodically exploring new regions to escape stagnation. Collectively, these analyses provide a mechanistic explanation for the accelerated and superior convergence of LB-MCTS observed in Figure 4.

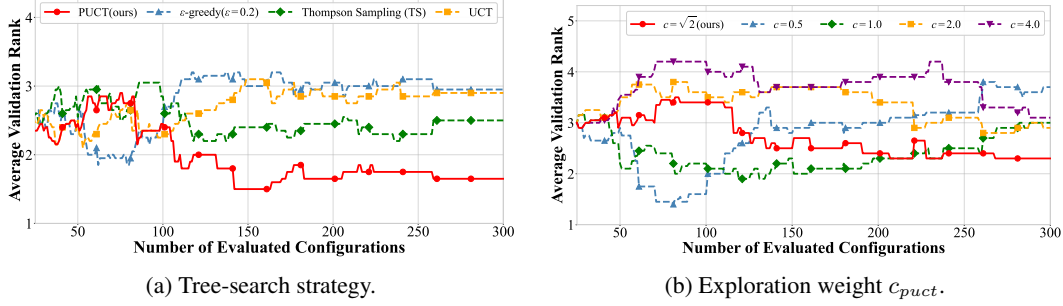


Figure 15: Ablations of the PUCT-based tree search.

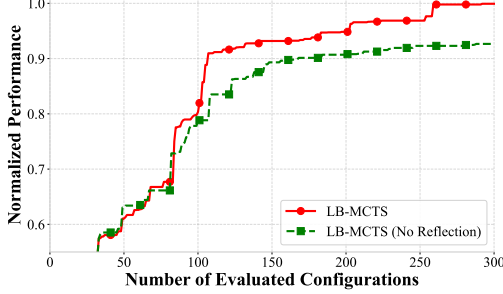


Figure 16: Ablation study on Reflection over 10 datasets.

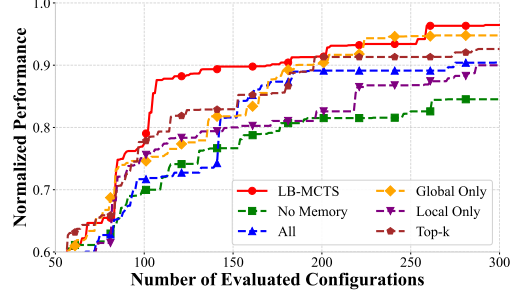


Figure 17: Ablation study on STM over 10 datasets.

C.10 Ablation of PUCT-Based Tree Search

In this appendix, we conduct ablation studies on 10 representative datasets to first evaluate whether PUCT is an appropriate tree-search policy for LB-MCTS, and then analyze the sensitivity of its exploration weight c_{puct} .

Choice of tree-search policy. We adopt PUCT because it combines empirical value estimates with prior-guided exploration, making it well suited for structured CASH search. As shown in Figure 15a, PUCT achieves the best average rank (1.65), outperforming Thompson Sampling (2.50), UCT (2.90), and ϵ -greedy (2.95). The advantage comes from PUCT’s ability to bias exploration toward globally promising branches while still preserving sufficient search diversity. In contrast, UCT ignores prior information and may spend evaluations on clearly weak branches. ϵ -greedy tends to over-commit to the current best branch, with exploration occurring only through random perturbations, making it vulnerable to local optima. Thompson Sampling provides stochastic exploration, but lacks the explicit prior-weighted control used by PUCT, leading to less stable search in our setting.

Exploration-weight sensitivity. We further evaluate the sensitivity to the exploration weight c_{puct} . As shown in Figure 15b, small values lead to over-exploitation and premature convergence, whereas large values over-emphasize exploration and slow down refinement. The results show that intermediate values are generally stable, and our default choice $c_{puct} = \sqrt{2}$ maintains strong performance throughout the search process.

C.11 Ablation Study: Reflection Mechanism

To assess the contribution of the LLM’s self-reflection capability, we conducted an ablation study by deactivating the reflection module (denoted as No Reflection) while keeping all other components unchanged. We aggregated the *normalized best-achieved validation performance* across the 10 selected datasets to analyze the global trend. As observed in Figure 16, (i) LB-MCTS consistently maintains a higher normalized score throughout the optimization process. (ii) The performance gap becomes more pronounced in the later stages (e.g., after 110 iterations). Finally, LB-MCTS achieves a superior **average validation rank of 1.1**, compared to 1.7 for the No Reflection baseline. This indicates that while both methods start similarly, the reflection mechanism effectively enables the framework to learn from historical failures and successes. It acts as a trajectory-level state summarizer to improve planning in the long run rather than a reactive proposal.

C.12 Ablation Study: Selective Tuning Memory

In this appendix, we validate the effectiveness of our Selective Tuning Memory (STM) mechanism. Unlike prior methods that mix history from different algorithms, a key prerequisite of our framework is the isolation of optimization history by algorithm to prevent cross-algorithm noise. Building on this cleaner, algorithm-specific history, we adapt and thereby strengthen retrieval strategies from existing LLM-based optimizers for fair comparison: (1) All: including the entire history of the current algorithm, mimicking the strategy of BORA and LLAMBO; (2) Top- k : retrieving only the best-performing trials, mimicking OPRO (here we set $k = 20$ to match the context length of OPRO). Next, we ablate the components of our STM design: (3) Global Only: retaining only the Pareto-selected global memory; (4) Local Only: retaining only the ancestral trajectory. Note that *all variants operate on per-algorithm histories (no cross-algorithm mixing)*, so differences come purely from the retrieval strategy within each algorithm. Finally, (5) we include a lower bound baseline, No Memory, which provides zero historical trials.

Figure 17 plots the normalized performance averaged over 10 datasets (detailed in Appendix C.8). Five observations can be concluded: (i) LB-MCTS (full STM) outperforms all variants, achieving the **best final average rank of 1.80**. (ii) We observe that strategies relying solely on high-performing samples—Global Only (Rank 2.30) and Top- k (Rank 2.80)—perform reasonably well but are sub-optimal. This indicates that while knowing “where the best solutions are” is useful, lacking the context of “how we got here” (trajectory) limits the LLM’s reasoning ability. In particular, trajectory information may help the LLM (a) avoid repeatedly moving back into regions that have already been explored and found unpromising, and (b) infer trends between parameter changes and performance changes—analogous to a coarse, history-based gradient signal—both of which are absent when only Global Memory are shown. (iii) Global Only outperforms Top- k , confirming that retrieving trials based on the Pareto frontier of similarity and performance provides more relevant guidance to the current expansion task than simply selecting based on raw performance alone. (iv) All (Rank 3.60) exposes the entire history and therefore includes the trajectory, but suffers from severe context pollution—many mediocre or redundant trials dilute the useful signal and consume the prompt budget. (v) Finally, Only Local (Rank 4.30) and No Memory (Rank 5.40) yield the poorest results, underscoring that without a set of high-quality global references, the search struggles to efficiently locate optimal regions.

Appendix References

- [51] Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, et al. Many-shot jailbreaking. *Advances in Neural Information Processing Systems*, 37:129696–129742, 2024.
- [52] Sercan Ö Arik and Tomas Pfister. Tabnet: Attentive interpretable tabular learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 6679–6687, 2021.
- [53] Yijun Bian and Huanhuan Chen. When does diversity help generalization in classification ensembles? *IEEE Transactions on Cybernetics*, 52(9):9059–9075, 2021.
- [54] Adam D Bull. Convergence rates of efficient global optimization algorithms. *Journal of Machine Learning Research*, 12(10), 2011.
- [55] Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *Proceedings of the twenty-first international conference on Machine learning*, page 18, 2004.
- [56] DeepSeek-AI. DeepSeek-V3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- [57] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. *Advances in neural information processing systems*, 28, 2015.
- [58] Google DeepMind. Gemini 2.0 flash, 2025. URL <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash>.

- [59] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. *Advances in neural information processing systems*, 34: 18932–18943, 2021.
- [60] Yang Li, Yu Shen, Wentao Zhang, Ce Zhang, and Bin Cui. Volcanoml: speeding up end-to-end automl via scalable search space decomposition. *The VLDB Journal*, 32(2):389–413, 2023.
- [61] Toni JB Liu, Nicolas Boullé, Raphaël Sarfati, and Christopher Earls. Llms learn governing principles of dynamical systems, revealing an in-context neural scaling law. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 15097–15117, 2024.
- [62] OpenAI. Gpt-4o model card., 2024. URL <https://platform.openai.com/docs/models/gpt-4o>.
- [63] Joaquin Vanschoren, Jan N Van Rijn, Bernd Bischl, and Luis Torgo. Openml: networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014.
- [64] Beicheng Xu, Wei Liu, Keyao Ding, Yupeng Lu, and Bin Cui. Pseo: Optimizing post-hoc stacking ensemble through hyperparameter tuning. *arXiv preprint arXiv:2508.05144*, 2025.
- [65] Wentao Zhang, Jiawei Jiang, Yingxia Shao, and Bin Cui. Efficient diversity-driven ensemble for deep neural networks. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, pages 73–84. IEEE, 2020.
- [66] Zhi-Hua Zhou, Jianxin Wu, and Wei Tang. Ensembling neural networks: many could be better than all. *Artificial intelligence*, 137(1-2):239–263, 2002.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Section 1 summarize the scope and contributions of LB-MCTS, including tree-structured LLM-BO collaboration, STM, dynamic proposer selection, and evaluation on 104 AMLB datasets. These claims are supported by Sections 3.1, 3.2 and 3.3.3 and the experimental results in Section 4.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We discuss limitations in Appendix A.1, including noisy validation feedback and dependence on the underlying LLM, and analyze API cost and wall-clock overhead in Appendix C.7.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The global convergence theorem is stated in Theorem 3.1, and Appendix A.3 provides the full assumptions, finite-time regret analysis, acceleration characterization, and proof.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The experimental setup in Section 4 specifies the baselines, datasets, metrics, data splits, evaluation budget, LLM backbone, and key hyperparameters. Further reproducibility details are provided in Appendix C.1 (search space), Appendix C.3 (baseline implementations and compute environment), and Appendix B.1 (LLM prompt templates).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide the code as supplementary material with instructions for reproducing the main experiments. The datasets are publicly available through AMLB/OpenML

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The experimental setup is described in Section 4, including datasets, metrics, data splits, baselines, budget, and key hyperparameters.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We assess statistical significance for the main benchmark using a Critical Difference diagram with the Nemenyi post-hoc test over 104 AMLB datasets, as shown in Figure 2b. A similar CD analysis for post-hoc ensemble performance is provided in Appendix C.6.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We report the compute environment in Appendix C.3 and the evaluation budget in Section 4. Appendix C.7 further analyzes API cost and wall-clock overhead.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research uses public benchmark datasets and does not involve human subjects, private data collection, or deployment in high-risk settings. To the best of our knowledge, the work conforms to the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We discuss broader impacts in Appendix A.1.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release pretrained generative models, scraped datasets, or other assets with high misuse risk.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We properly cite the existing assets used in this work, including AMLB/OpenML datasets, prior AutoML systems and baselines, and LLM backbones. Public datasets are accessed through AMLB/OpenML under their original licenses and terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We release the implementation of LB-MCTS as supplementary material.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This work does not involve crowdsourcing, user studies, human-subject experiments, or human data collection.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This work does not involve human-subject experiments, crowdsourcing, user studies, or human data collection.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLM usage is a core component of the proposed method. We describe the LLM proposer in Section 3.3.2, and the LLM backbone used in experiments in Section 4. Prompt designs for hyperparameter tuning and reflection are provided in Appendix B.1 and Appendix B.2.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.